UNIVERSIDAD EL BOSQUE

AVANCE 1 — NÚCLEO TRANSACCIONAL EN POSTGRESQL
AVANCE 2 — CAPA DOCUMENTAL (MONGODB) Y FUNDAMENTOS TEÓRICOS

CINE AURORA



TAMAÑO DE GRUPO:
2 INTEGRANTES

INTEGRANTES:

YEISON ALEJANDRO SILVA RODRIGUEZ
LEIDY VANESSA GIL DAZA

BASES DE DATOS 2

DOCENTE:

FRANCISCO BARGUIL

12 DE NOVIEMBRE DE 2026
BOGOTA D.C

Tabla de contenido

AVANCE 1 — NÚCLEO TRANSACCIONAL EN POSTGRESQL 4
1. Identidad de la empresa de cine 4
2. Modelo entidad-relación 5
2.1 Relaciones y cardinalidades 5
2.2 Relaciones muchos a muchos resueltas 6
3. Modelo relacional (3FN) 7
3.1 Justificación de la normalización 8
3.2 Decisiones de diseño sobre datos derivados 9
4. Script SQL de creación 10
4.1 Restricciones de integridad por tabla 10
4.2 Índices básicos 11
5. Datos de prueba 12
6. Triggers 14
Trigger 1: trg_calcular_precio 14
Trigger 2: trg_validar_capacidad 15
Trigger 3: trg_validar_taquillero 18
Trigger 4: trg_validar_horario_funcion 19
Trigger 5: trg_validar_contrato 21
Trigger 6: trg_validar_mantenimiento_sala 22
Trigger 7: trg_validar_inactivar_staff 23
Trigger 8: trg_normalizar_espectador 24
7. Procedimientos almacenados 26
Procedimiento 1: registrar_venta 26
Procedimiento 2: reporte_ventas_funcion 30
Procedimiento 3: actualizar_estado_funciones 33
8. Consultas SQL 35
Consulta 1: Rentabilidad por película 35
Consulta 2: Ocupación por sala 36
Consulta 3: Espectadores que gastan más que el promedio 37
9. Repositorio y control de versiones 40
9.1 Commits por integrante 41
AVANCE 2 — Capa Documental (MongoDB) y Fundamentos Teóricos 43
1. Introducción 43
2. Arquitectura de persistencia poliglota 43
2.1 Diagrama de arquitectura políglota 44
3. Diseño de las colecciones 46
3.1 fichas_tecnicas_peliculas 46
3.2 requisitos_proyeccion 47
3.3 resenas_peliculas 48

3.4 reportes_incidentes 50
4. Validación con $jsonSchema 52
5. Scripts y datos de prueba 53
6. Consultas en MongoDB 54
6.1 Consultas con find 54
6.2 Consultas con aggregation pipeline 55
7. Capturas de ejecución 57
7.1 Creación de las colecciones (script 01) 57
7.2 Carga de datos de prueba (script 02) 58
7.3 Consultas y agregaciones (script 03) 59
7.4 Agregación 2: incidentes por tipo (script 03) 59
7.5 Agregación 3: funciones por tipo de proyección (script 03) 60
8. Justificación teórica ACID, BASE Y CAP 60
8.1 ACID en PostgreSQL 60
8.2 BASE en MongoDB 61
8.3 Teorema CAP 62
9. Repositorio y control de versiones 64
9.1 Commits por integrante 65
10. Conclusiones 66

AVANCE 1 — NÚCLEO TRANSACCIONAL EN POSTGRESQL

1. Identidad de la empresa de cine

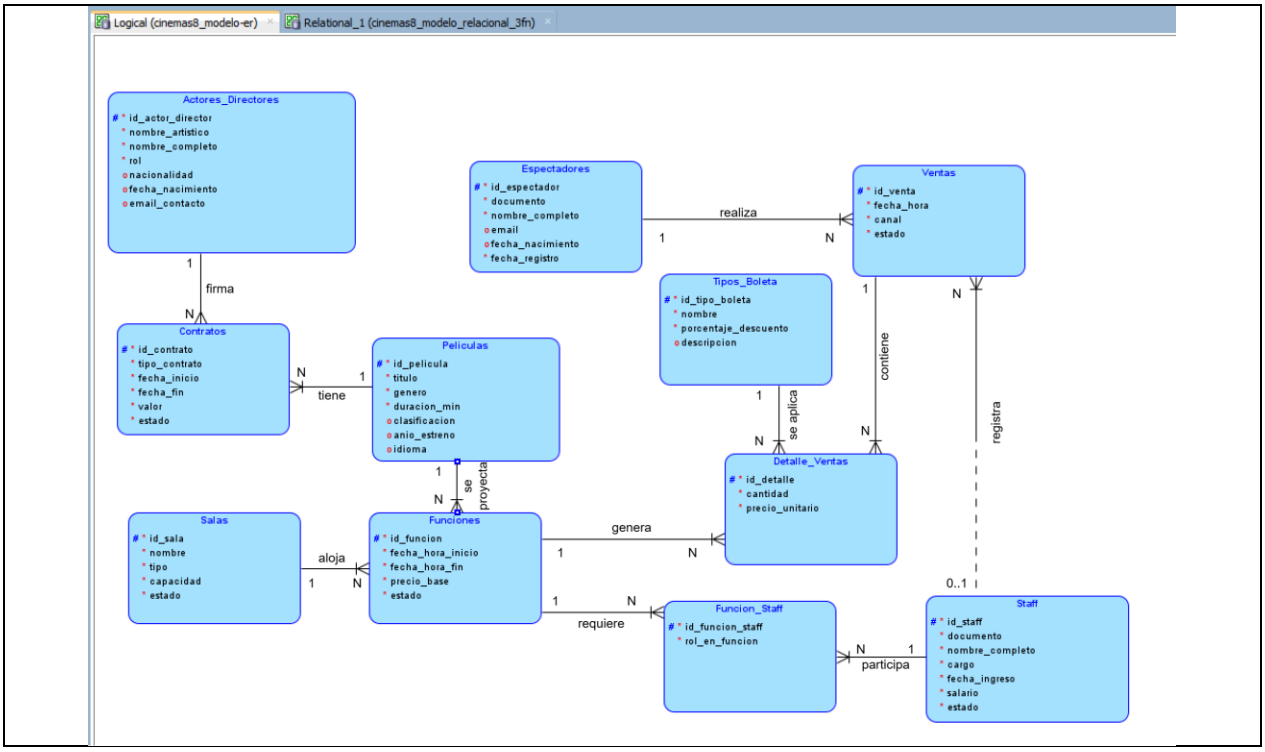
Cine Aurora (grupo Cinemas 8) es una empresa de cine de Bogotá. Pasamos películas de cine de autor, clásico e independiente y también algunos estrenos de aventura y ciencia ficción. Además, hacemos presentaciones, charlas y ciclos con actores y directores, por eso la base de datos maneja también los contratos con ellos y no solo la venta de boletas.

Nombre	**Cine Aurora**
Enfoque de programación	Cine de autor, clásico e independiente (drama, documental, animación, romance y musical), con algunos estrenos de aventura, terror y ciencia ficción. También hay presentaciones, charlas y ciclos con actores y directores.
Ciudad	Bogotá D.C., Colombia
Modalidad de operación	Mixta es decir que se vende en la taquilla y también por internet.
Capacidad estimada	1.200 espectadores, que es la suma de la capacidad de las 8 salas
Número de salas	8 salas: 2 VIP (Aurora de 80 y Boreal de 100), 2 en 3D (Eclipse de 150 y Fénix de 160) y 4 en 2D (Cenit y Delta de 120, Galaxia de 220 y Horizonte de 250)
Actores/directores registrados	40 artistas en la base de datos.
Particularidades del negocio	-Tenemos tres tipos de salas (2D, 3D y VIP), y cada una maneja un precio base para sus funciones. -Las boletas se dividen en 4 categorías: General (sin descuento), Estudiante (20% de descuento), Tercera edad (30% de descuento) y Matiné (25% de descuento). -Los contratos con los actores y directores pueden ser de tres tipos: Presentación, Charla o Ciclo. -Las ventas presenciales en la taquilla siempre van asociadas a un taquillero, mientras que las compras online se hacen de forma directa sin requerir personal de taquilla. -A cada función se le asigna personal de staff cumpliendo roles específicos: responsable, proyección o acomodación. -El sistema cuenta con validaciones estrictas para evitar sobreventas (no se pueden vender más boletas de la capacidad de la sala) y para impedir que se crucen dos funciones al mismo tiempo en una misma sala.
Logo	

2. Modelo entidad-relación

Tenemos las 8 entidades que pide el avance (Actores_Directores, Contratos, Salas, Funciones, Tipos_Boleta, Ventas, Espectadores y Staff). Para poder llegar a la tercera forma normal tuvimos que agregar la tabla películas, y para las relaciones de muchos a muchos creamos dos tablas intermedias: detalle_ventas y funcion_staff. La tabla contratos también cumple ese papel, porque une a los actores y directores con las películas.

Figura 1. Diagrama entidad-relación



2.1 Relaciones y cardinalidades

Relación	Cardinalidad	Descripción
actores_directores - contratos	1: N	Un actor o director puede tener varios contratos, pero cada contrato es de un solo artista.
películas - contratos	1: N	Una película puede tener varios contratos.
salas - funciones	1: N	Una sala tiene muchas funciones y cada función es en una sola sala.
películas - funciones	1: N	Una película se pasa en muchas funciones.
espectadores - ventas	1: N	Un espectador puede hacer muchas ventas.

Relación	Cardinalidad	Descripción
staff - ventas	0..1: N	Un taquillero registra muchas ventas en la taquilla. Las ventas en línea no tienen taquillero.
ventas - detalle_ventas	1: N	Una venta tiene una o más líneas de detalle.
funciones - detalle_ventas	1: N	Una función puede aparecer en muchas líneas de venta.
tipos_boleta - detalle_ventas	1: N	Un tipo de boleta se usa en muchas líneas de venta.
funciones - funcion_staff	1: N	Una función tiene varios empleados asignados.
staff - funcion_staff	1: N	Un empleado trabaja en muchas funciones.

2.2 Relaciones muchos a muchos resueltas

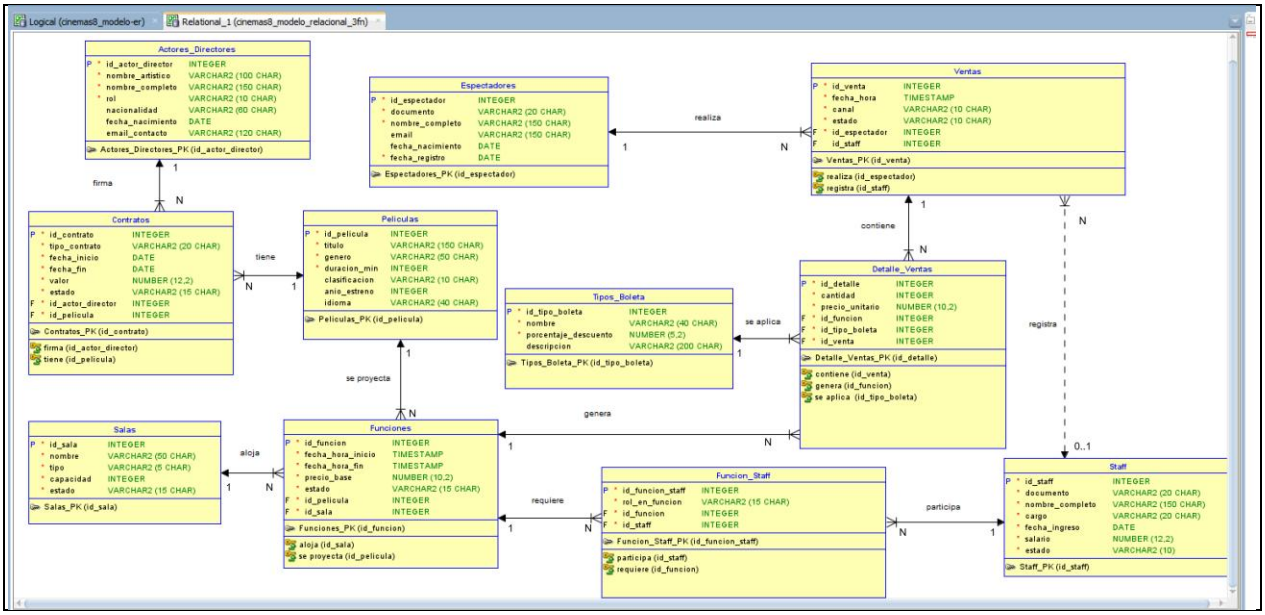
- actores_directores y películas, por medio de contratos con tipo, fechas, valor y estado.
- ventas y funciones, por medio de detalle_ventas con el tipo de boleta, la cantidad y el precio unitario
- funciones y staff, por medio de funcion_staff con el rol del empleado en la función.

3. Modelo relacional (3FN)

PK es llave primaria, FK es llave foránea y UQ es valor único.

Relación	Atributos	Llaves
actores_directores	id_actor_director, nombre_artistico, nombre_completo, rol, nacionalidad, fecha_nacimiento, email_contacto	PK: id_actor_director
peliculas	id_pelicula, titulo, genero, duracion_min, clasificacion, anio_estreno, idioma	PK: id_pelicula
salas	id_sala, nombre, tipo, capacidad, estado	PK: id_sala; UQ: nombre
staff	id_staff, documento, nombre_completo, cargo, fecha_ingreso, salario, estado	PK: id_staff; UQ: documento
tipos_boleta	id_tipo_boleta, nombre, porcentaje_descuento, descripcion	PK: id_tipo_boleta; UQ: nombre
espectadores	id_espectador, documento, nombre_completo, email, fecha_nacimiento, fecha_registro	PK: id_espectador; UQ: documento
contratos	id_contrato, id_actor_director, id_pelicula, tipo_contrato, fecha_inicio, fecha_fin, valor, estado	PK: id_contrato; FK: id_actor_director, id_pelicula
funciones	id_funcion, id_sala, id_pelicula, fecha_hora_inicio, fecha_hora_fin, precio_base, estado	PK: id_funcion; FK: id_sala, id_pelicula
ventas	id_venta, id_espectador, id_staff, fecha_hora, canal, estado	PK: id_venta; FK: id_espectador, id_staff (opcional)
detalle_ventas	id_detalle, id_venta, id_funcion, id_tipo_boleta, cantidad, precio_unitario	PK: id_detalle; UQ: (id_venta, id_funcion, id_tipo_boleta); FK: id_venta, id_funcion, id_tipo_boleta
funcion_staff	id_funcion_staff, id_funcion, id_staff, rol_en_funcion	PK: id_funcion_staff; UQ: (id_funcion, id_staff); FK: id_funcion, id_staff

Figura 2. Modelo relacional



3.1 Justificación de la normalización

Primera forma normal (1FN): todos los campos tienen un solo valor, no hay listas ni columnas repetidas. Por ejemplo, las líneas de una venta no están como columnas dentro de ventas, sino como filas en detalle_ventas.

Segunda forma normal (2FN): todas las tablas tienen una llave primaria de un solo campo (un id que se genera solo), así que no hay dependencias parciales. En las tablas intermedias además dejamos un valor único sobre la combinación (por ejemplo, id_venta, id_funcion e id_tipo_boleta) para que no se repitan líneas.

Tercera forma normal (3FN): ningún campo que no sea llave depende de otro campo que tampoco lo sea. Por ejemplo:

- Los datos de la película (título, género, duración, clasificación) dependen de la película y no de la función ni del contrato, por eso están en películas.
- El porcentaje de descuento depende del tipo de boleta, entonces está en tipos_boleta y no se repite en cada venta.
- La capacidad y el tipo de sala dependen de la sala y no de la función, por eso están en salas.
- Los datos del espectador y del empleado están en sus propias tablas y las ventas solo los llaman con una llave foránea.

3.2 Decisiones de diseño sobre datos derivados

- precio_unitario (en detalle_ventas) lo guardamos como dato histórico, porque es lo que realmente se cobró y no debería cambiar si después cambia el precio base o el descuento. Lo calcula el trigger 1. El total de una línea se saca con cantidad por precio_unitario y no se guarda.
- fecha_hora_fin (en funciones) se podría calcular con la hora de inicio más la duración de la película, pero la dejamos guardada para poder revisar los cruces de horario. La calcula el trigger 4, así que no queda con un valor incorrecto.

4. Script SQL de creación

El archivo 01_creacion_tablas.sql crea las 11 tablas con sus llaves primarias, llaves foráneas, restricciones (UNIQUE, NOT NULL, DEFAULT y CHECK) e índices básicos. Empieza borrando las tablas si ya existen (DROP TABLE IF EXISTS ... CASCADE), así que se puede volver a ejecutar desde cero.

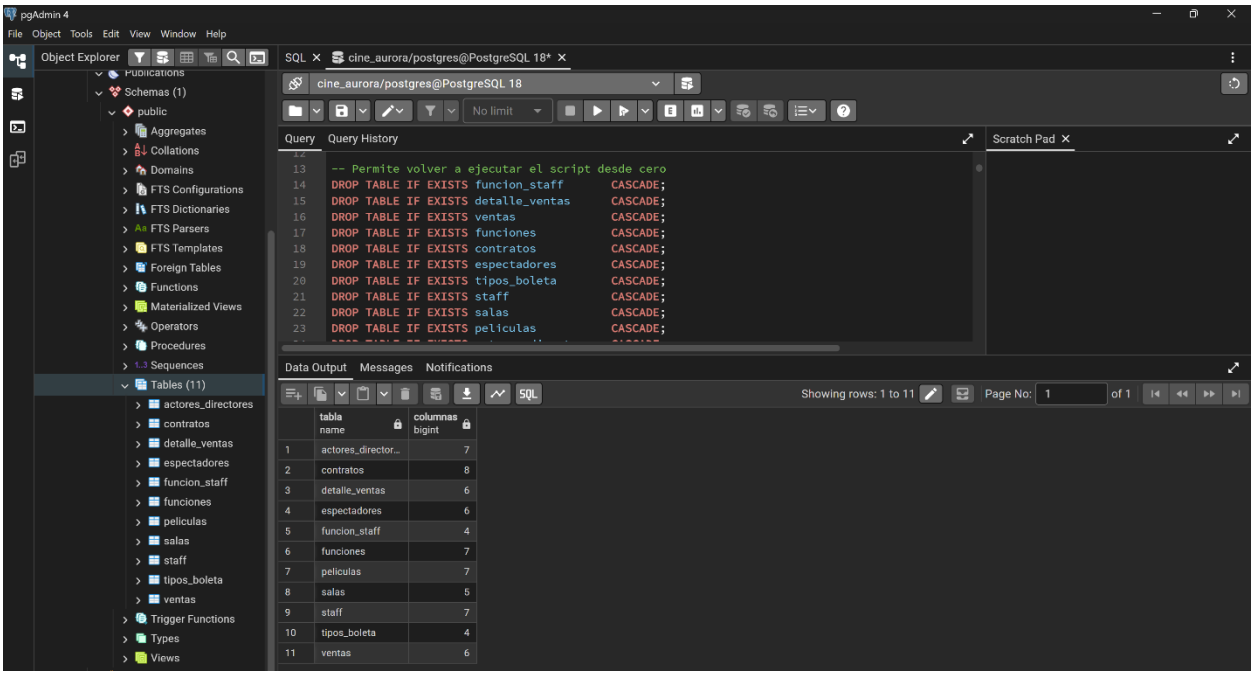
4.1 Restricciones de integridad por tabla

Tabla	Restricciones principales
actores_directores	El rol solo puede ser Actor, director o Ambos; los nombres no pueden estar vacíos; la fecha de nacimiento debe ser desde 1900; el correo debe tener formato válido.
peliculas	El título no puede estar vacío; la duración va entre 1 y 600 minutos; el año de estreno va entre 1888 y 2100.
salas	El nombre es único; el tipo solo puede ser 2D, 3D o VIP; la capacidad va entre 1 y 250; el estado es Activa o Mantenimiento.
staff	El documento es único; el cargo es Taquillero, Acomodador, Proyeccionista o Gerente; el salario debe ser mayor que 0; el estado es Activo o Inactivo; la fecha de ingreso debe ser desde 1990.
tipos_boleta	El nombre es único; el porcentaje de descuento va entre 0 y 100.
espectadores	El documento es único; el nombre no puede estar vacío; el correo debe ser válido; la fecha de nacimiento debe ser desde 1900.
contratos	El tipo es Presentación, Charla o Ciclo; la fecha de fin no puede ser antes de la de inicio; el valor no puede ser negativo; el estado es Vigente, Finalizado o Cancelado; llaves foráneas a actores_directores y películas.
funciones	La hora de fin debe ser mayor que la de inicio; el precio base no puede ser negativo; el estado es Programada, En curso, Finalizada o Cancelada; llaves foráneas a salas y películas.
ventas	El canal es Presencial u Online; el estado es Pagada o Anulada; una venta presencial siempre tiene empleado; llaves foráneas a espectadores y staff.
detalle_ventas	La cantidad debe ser mayor que 0; el precio unitario no puede ser negativo; no se repite la misma línea (venta, función y tipo de boleta); llaves foráneas a ventas, funciones y tipos_boleta.
funcion_staff	El rol es responsable, Proyección o Acomodación; un empleado no se repite en la misma función; llaves foráneas a funciones y staff.

4.2 Índices básicos

- Índices únicos sobre (email) en espectadores y actores_directores, para que no se repita un correo, aunque cambien las mayúsculas.
- idx_funciones_sala_inicio (id_sala, fecha_hora_inicio), que se usa para revisar los cruces de horario de una sala.
- Índices sobre las llaves foráneas de ventas, detalle_ventas, contratos, funciones y funcion_staff, para que los reportes y la validación de capacidad sean más rápidos.

Captura 1. Ejecución del script 01_creacion_tablas.sql

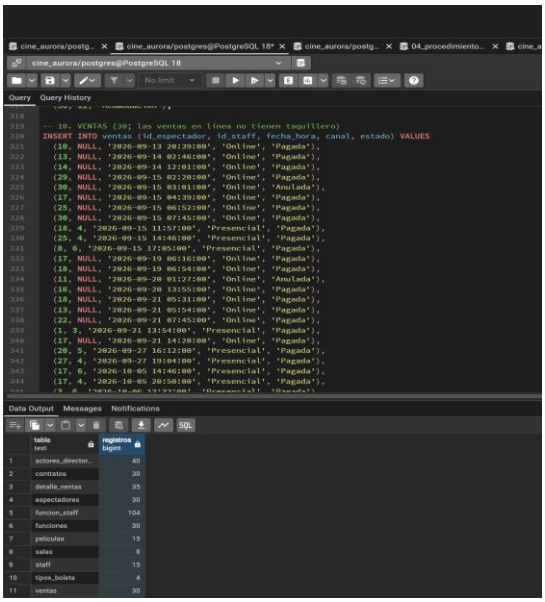


5. Datos de prueba

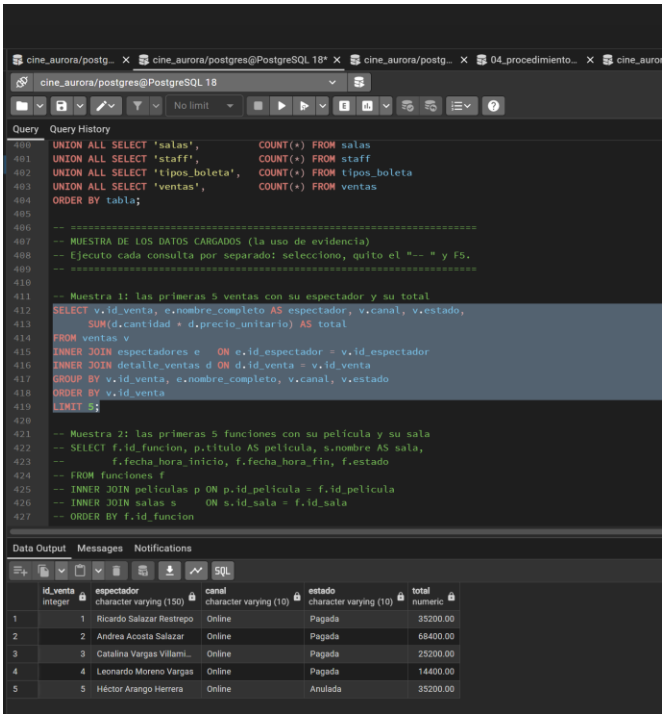
El archivo 02_datos_prueba.sql se ejecuta después del 01. Primero limpia las tablas y después inserta los datos, las ventas apuntan a espectadores que existen, las funciones tienen película y sala válidas, el detalle de ventas tiene el precio con el descuento del tipo de boleta y cada función tiene su personal asignado.

Tabla	Registros
actores_directores	40
espectadores	30
ventas	30
funciones	30 (20 finalizadas y 10 programadas)
películas	15
salas	8
staff	15
tipos_boleta	4
contratos	30
funcion_staff	104
detalle_ventas	35

Captura 2. Ejecución del script 02_datos_prueba.sql



Captura 3. Muestra de los datos cargados



```
UNION ALL SELECT 'salas',          COUNT(*) FROM salas
UNION ALL SELECT 'staff',          COUNT(*) FROM staff
UNION ALL SELECT 'tipos_boleta',   COUNT(*) FROM tipos_boleta
UNION ALL SELECT 'ventas',         COUNT(*) FROM ventas
ORDER BY tabla;

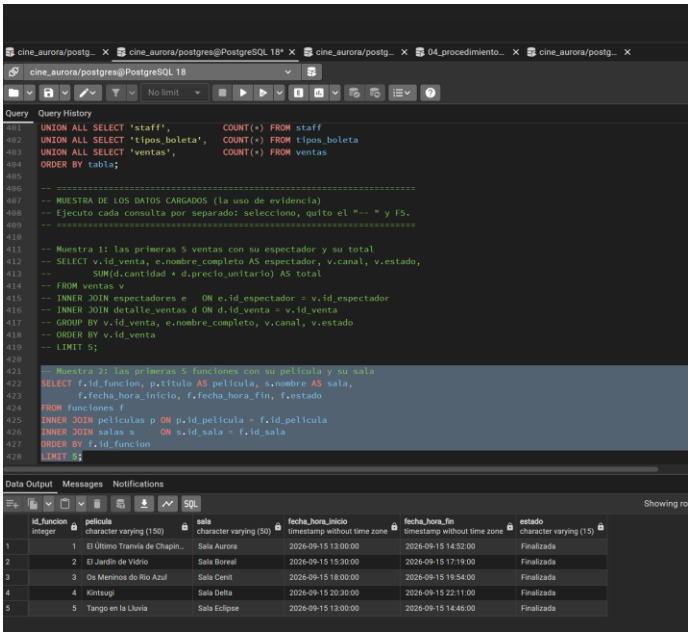
-- ======================================================================
-- MUESTRA DE LOS DATOS CARGADOS (la uso de evidencia)
-- Ejecuto cada consulta por separado: selecciono, quito el "-- " y F5.
-- ======================================================================

-- Muestra 1: las primeras 5 ventas con su espectador y su total
SELECT v.id_venta, e.nombre_completo AS espectador, v.canal, v.estado,
       SUM(d.cantidad * d.precio_unitario) AS total
FROM ventas v
INNER JOIN espectadores e   ON e.id_espectador = v.id_espectador
INNER JOIN detalle_ventas d ON d.id_venta = v.id_venta
GROUP BY v.id_venta, e.nombre_completo, v.canal, v.estado
ORDER BY v.id_venta
LIMIT 5;

-- Muestra 2: las primeras 5 funciones con su película y su sala
-- SELECT f.id_funcion, p.titulo AS pelicula, s.nombre AS sala,
--        f.fecha_hora_inicio, f.fecha_hora_fin, f.estado
-- FROM funciones f
-- INNER JOIN peliculas p ON p.id_pelicula = f.id_pelicula
-- INNER JOIN salas s     ON s.id_sala = f.id_sala
-- ORDER BY f.id_funcion
```

Data Output | Messages | Notifications

id_venta integer	espectador character varying (150)	canal character varying (10)	estado character varying (10)	total numeric
1	1 Ricardo Salazar Restrepo	Online	Pagada	35200.00
2	2 Andrea Acosta Salazar	Online	Pagada	68400.00
3	3 Catalina Vargas Villami...	Online	Pagada	25200.00
4	4 Leonardo Moreno Vargas	Online	Pagada	14400.00
5	5 Héctor Arango Herrera	Online	Anulada	35200.00



```
UNION ALL SELECT 'staff',          COUNT(*) FROM staff
UNION ALL SELECT 'tipos_boleta',   COUNT(*) FROM tipos_boleta
UNION ALL SELECT 'ventas',         COUNT(*) FROM ventas
ORDER BY tabla;

-- ======================================================================
-- MUESTRA DE LOS DATOS CARGADOS (la uso de evidencia)
-- Ejecuto cada consulta por separado: selecciono, quito el "-- " y F5.
-- ======================================================================

-- Muestra 1: las primeras 5 ventas con su espectador y su total
-- SELECT v.id_venta, e.nombre_completo AS espectador, v.canal, v.estado,
--        SUM(d.cantidad * d.precio_unitario) AS total
-- FROM ventas v
-- INNER JOIN espectadores e   ON e.id_espectador = v.id_espectador
-- INNER JOIN detalle_ventas d ON d.id_venta = v.id_venta
-- GROUP BY v.id_venta, e.nombre_completo, v.canal, v.estado
-- ORDER BY v.id_venta
-- LIMIT 5;

-- Muestra 2: las primeras 5 funciones con su película y su sala
SELECT f.id_funcion, p.titulo AS pelicula, s.nombre AS sala,
       f.fecha_hora_inicio, f.fecha_hora_fin, f.estado
FROM funciones f
INNER JOIN peliculas p ON p.id_pelicula = f.id_pelicula
INNER JOIN salas s     ON s.id_sala = f.id_sala
ORDER BY f.id_funcion
LIMIT 5;
```

Data Output | Messages | Notifications

id_funcion integer	pelicula character varying (150)	sala character varying (50)	fecha_hora_inicio timestamp without time zone	fecha_hora_fin timestamp without time zone	estado character varying (15)
1	1 El Último Tranvía de Chapin...	Sala Aurora	2026-09-15 13:00:00	2026-09-15 14:52:00	Finalizada
2	2 El Jardín de Vidrio	Sala Boreal	2026-09-15 15:30:00	2026-09-15 17:19:00	Finalizada
3	3 Os Meninos do Rio Azul	Sala Cenit	2026-09-15 18:00:00	2026-09-15 19:54:00	Finalizada
4	4 Kintsugi	Sala Delta	2026-09-15 20:30:00	2026-09-15 22:11:00	Finalizada
5	5 Tango en la Lluvia	Sala Eclipse	2026-09-15 13:00:00	2026-09-15 14:46:00	Finalizada

En la Captura 2 se ve el INSERT del script y, debajo, el conteo de registros de las 11 tablas, que coincide con la tabla anterior. En la Captura 3 se consultan las primeras ventas con su espectador: por ejemplo, la venta 1 es de Ricardo Salazar Restrepo y suma $35.200. Esto permite comprobar que los datos cargados son los del archivo entregado.

6. Triggers

El archivo 03_triggers.sql tiene 8 triggers en 7 tablas, 6 de las 8 tablas pedidas (detalle_ventas, ventas, funciones, contratos, salas, staff y espectadores), así que la validación no queda toda en una sola tabla. Cada trigger tiene su función en PL/pgSQL y se crea con DROP TRIGGER IF EXISTS para poder ejecutar el script otra vez.

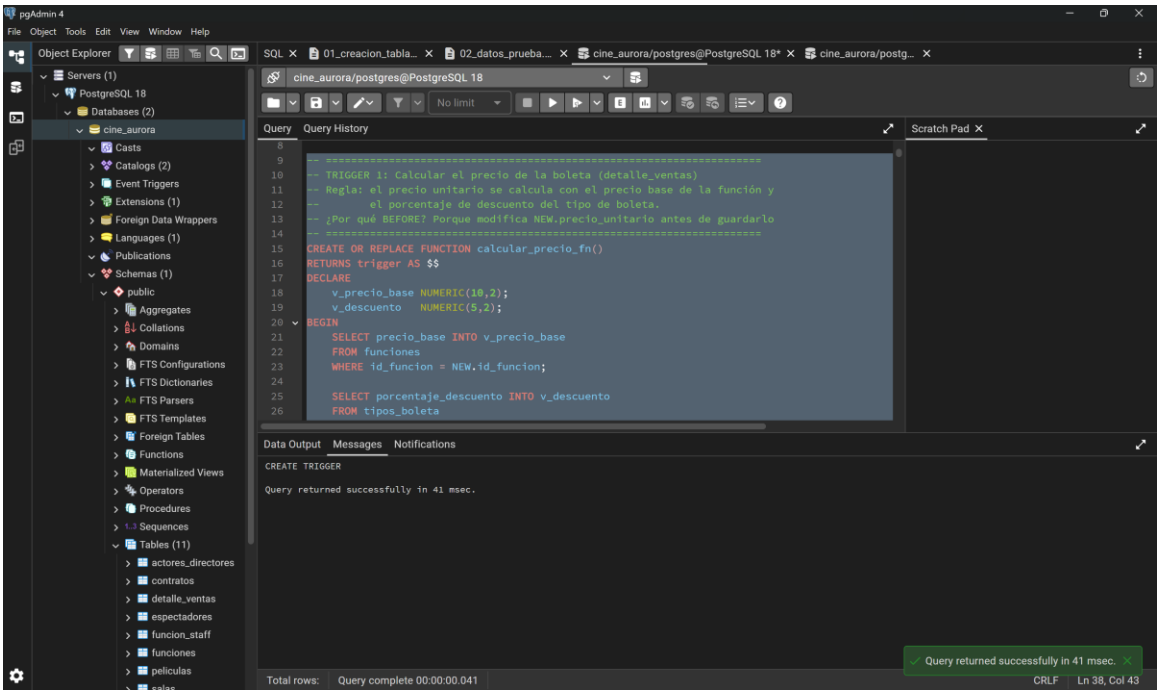
	Trigger	Tabla	Evento
1	trg_calcular_precio	detalle_ventas	BEFORE INSERT
2	trg_validar_capacidad	detalle_ventas	BEFORE INSERT OR UPDATE
3	trg_validar_taquillero	ventas	BEFORE INSERT
4	trg_validar_horario_funcion	funciones	BEFORE INSERT OR UPDATE
5	trg_validar_contrato	contratos	BEFORE INSERT OR UPDATE
6	trg_validar_mantenimiento_sala	salas	BEFORE UPDATE
7	trg_validar_inactivar_staff	staff	BEFORE UPDATE
8	trg_normalizar_espectador	espectadores	BEFORE INSERT OR UPDATE

Trigger 1: trg_calcular_precio

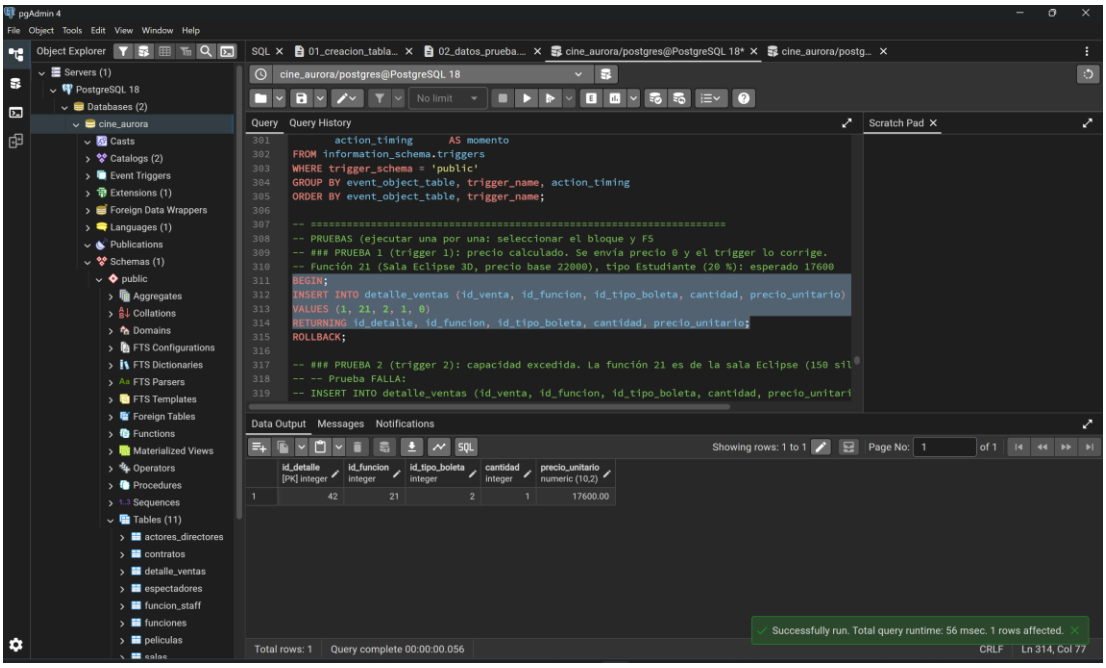
Tabla sobre la que actúa	detalle_ventas
Evento	BEFORE INSERT
Función asociada	calcular_precio_fn()
Regla de negocio que protege	El precio de la boleta siempre es el precio base de la función menos el descuento del tipo de boleta. Nadie puede escribir el precio a mano.

Busca el precio_base de la función y el porcentaje de descuento del tipo de boleta, y guarda en NEW.precio_unitario el precio base por (1 - descuento/100). Así no se pueden cobrar valores incorrectos.

Captura 4. Ejecución del trigger 1 (trg_calcular_precio)



Captura 5. Prueba del trigger 1 (trg_calcular_precio)



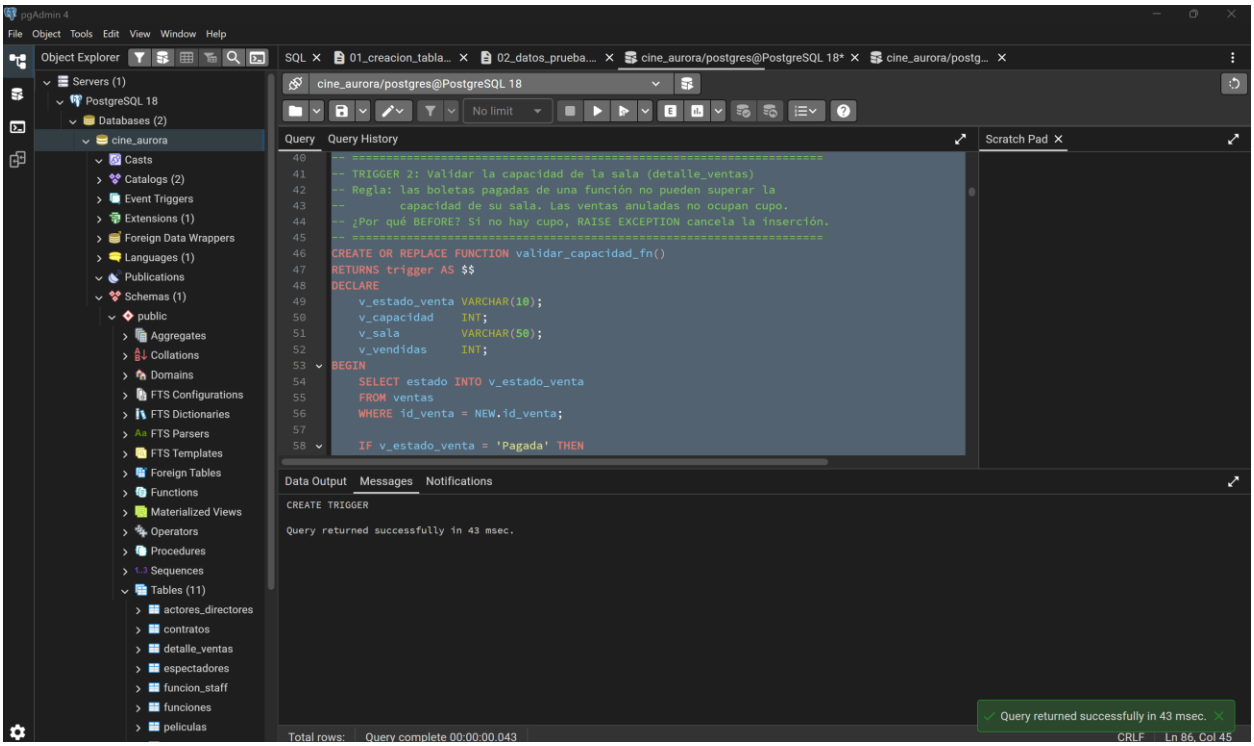
Trigger 2: trg_validar_capacidad

Tabla sobre la que actúa	detalle_ventas

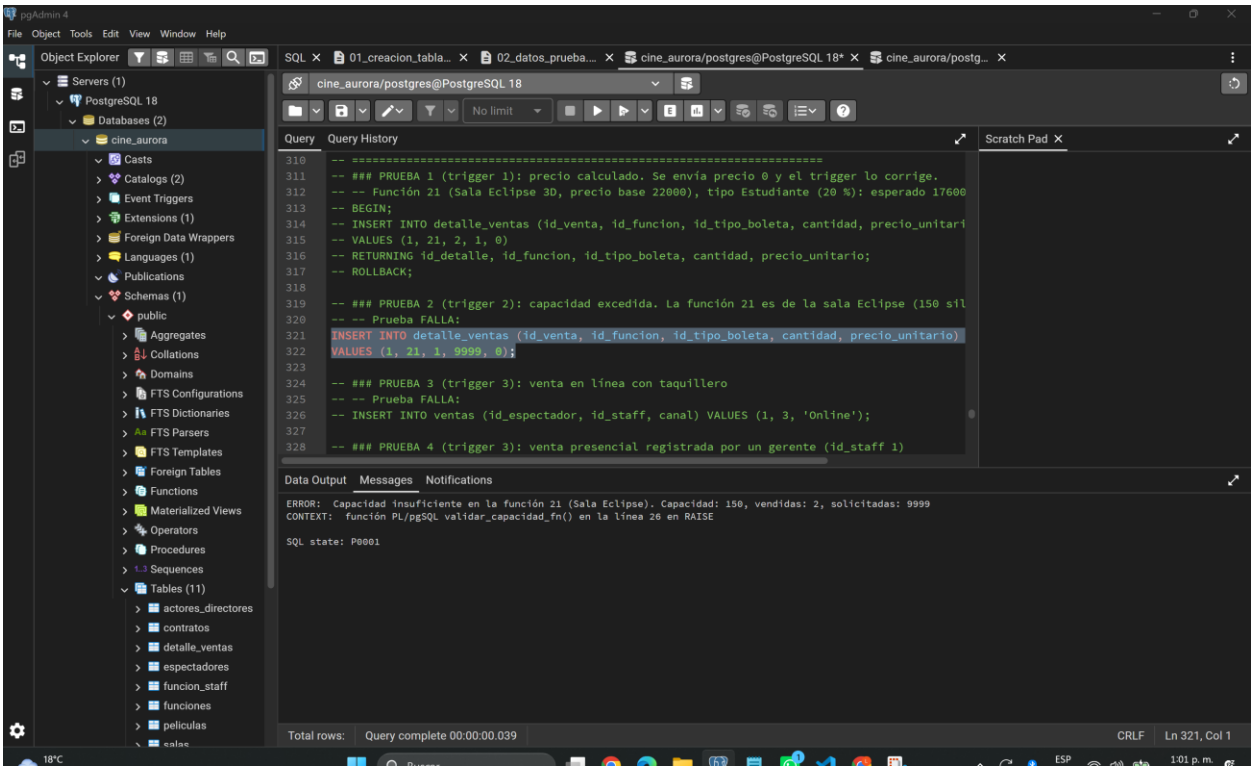
Evento	BEFORE INSERT OR UPDATE
Función asociada	validar_capacidad_fn()
Regla de negocio que protege	No se pueden vender más boletas que sillas tiene la sala.

Si la venta está Pagada, suma las boletas pagadas que ya tiene esa función, le suma las nuevas y las compara con la capacidad de la sala. Las ventas anuladas no cuentan, porque esas sillas quedan libres. Si se pasa de la capacidad, lanza un RAISE EXCEPTION con el cupo que queda.

Captura 6. Ejecución del trigger 2 (trg_validar_capacidad)



Captura 7. Prueba del trigger 2 (trg_validar_capacidad)

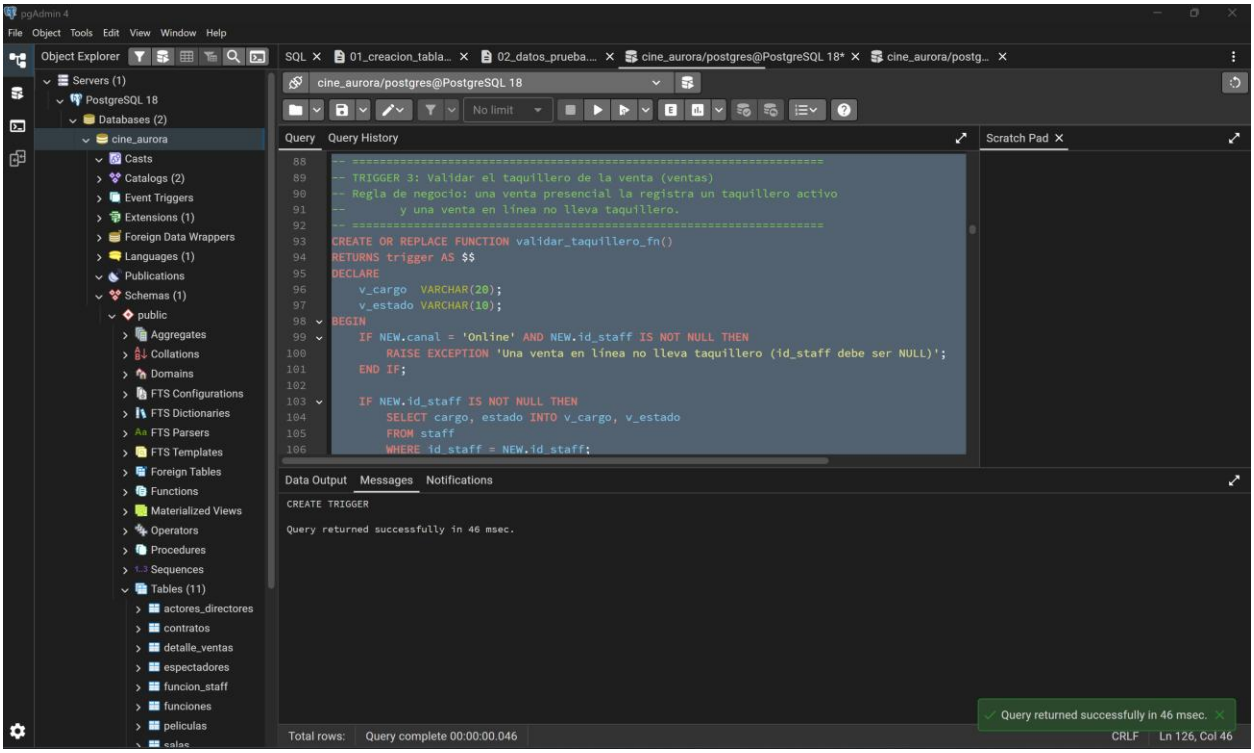


Trigger 3: trg_validar_taquillero

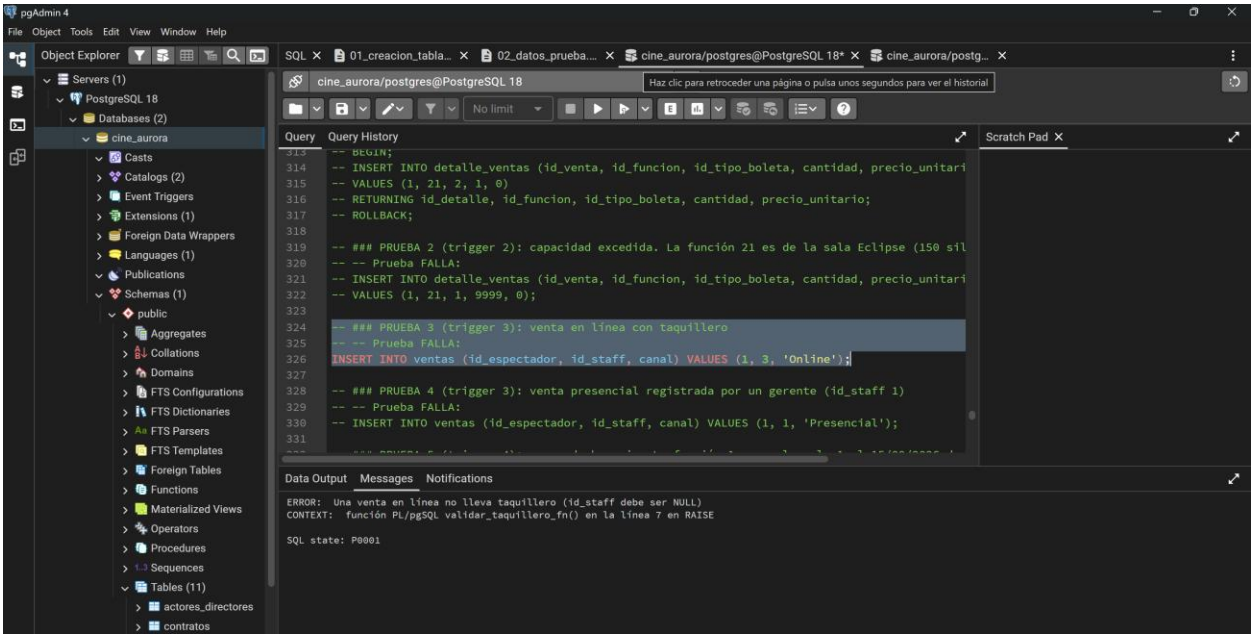
Tabla sobre la que actúa	ventas
Evento	BEFORE INSERT
Función asociada	validar_taquillero_fn()
Regla de negocio que protege	Una venta en línea no lleva empleado, y si lleva empleado tiene que ser un taquillero activo.

Rechaza las ventas Online que traen id_staff. Cuando la venta sí trae empleado, revisa que su cargo sea Taquillero y que esté Activo. Por ejemplo, un gerente o un empleado inactivo no puede vender en la taquilla.

Captura 8. Ejecución del trigger 3 (trg_validar_taquillero)



Captura 9. Prueba del trigger 3 (trg_validar_taquillero)

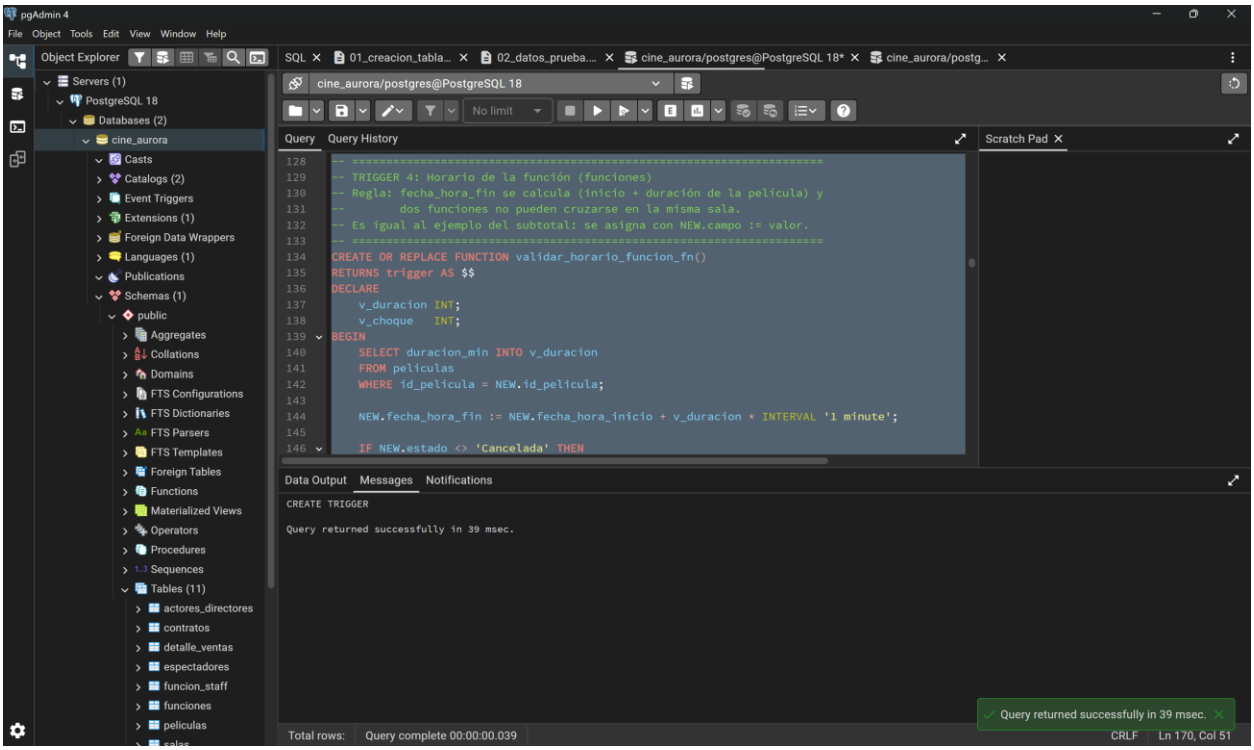


Trigger 4: trg_validar_horario_funcion

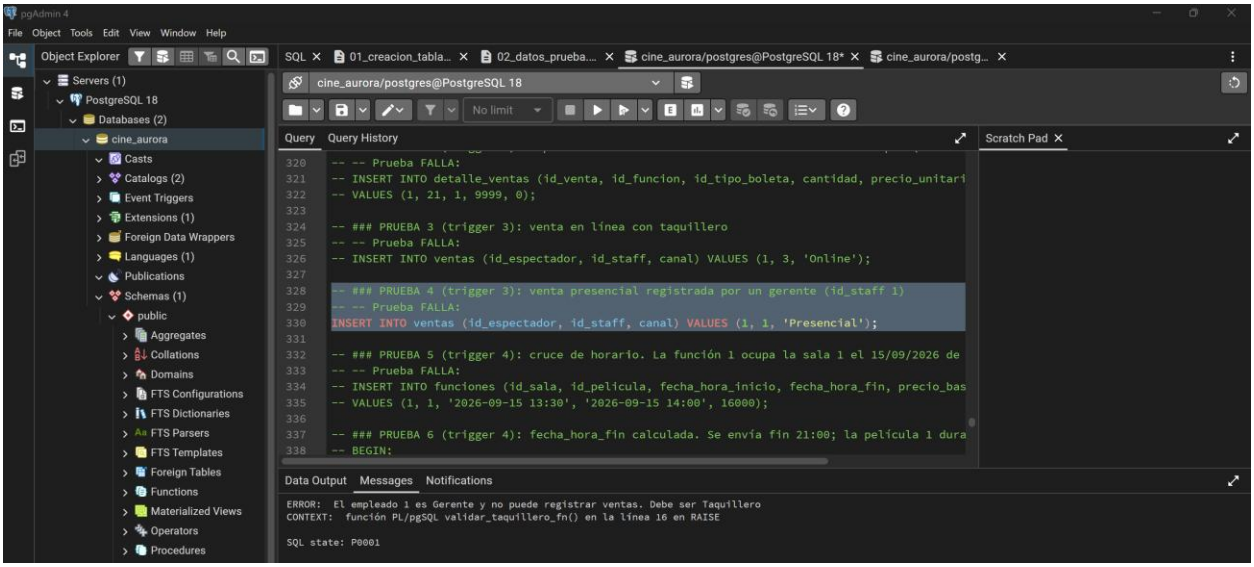
Tabla sobre la que actúa	funciones
Evento	BEFORE INSERT OR UPDATE
Función asociada	validar_horario_funcion_fn()
Regla de negocio que protege	La hora de fin es la hora de inicio más la duración de la película, y dos funciones no se pueden cruzar en la misma sala.

Calcula NEW.fecha_hora_fin con la duración de la película. Si la función no está cancelada, busca otra función no cancelada en la misma sala que se cruce en horario, y si la encuentra lanza un RAISE EXCEPTION diciendo con cuál choca.

Captura 10. Ejecución del trigger 4 (trg_validar_horario_funcion)



Captura 11. Prueba del trigger 4 (trg_validar_horario_funcion)

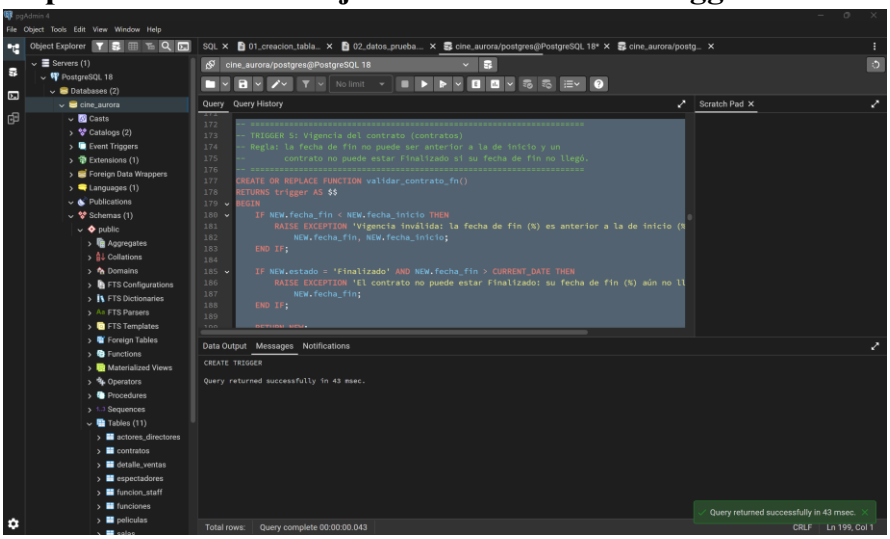


Trigger 5: trg_validar_contrato

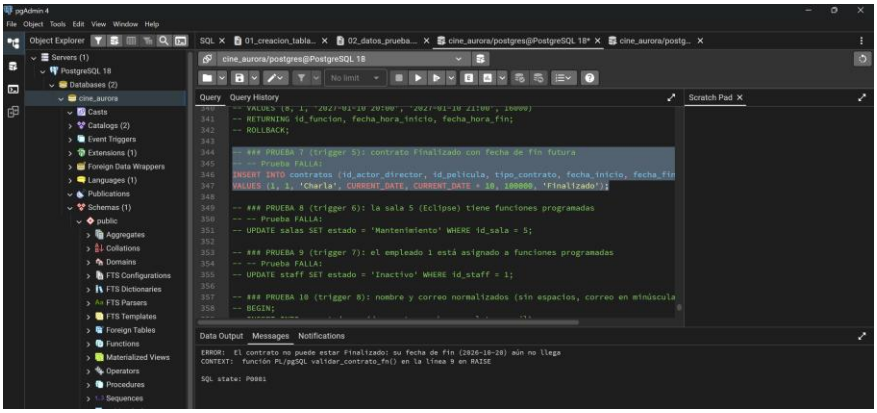
Tabla sobre la que actúa	contratos
Evento	BEFORE INSERT OR UPDATE
Función asociada	validar_contrato_fn()
Regla de negocio que protege	Un contrato no puede terminar antes de empezar, ni estar Finalizado si su fecha de fin todavía no llega.

Revisa que las fechas del contrato tengan sentido y que el estado corresponda con la fecha de hoy, para que no queden contratos con un estado que no es real.

Captura 12. Ejecución del trigger 5 (trg_validar_contrato)



Captura 13. Prueba del trigger 5 (trg_validar_contrato)

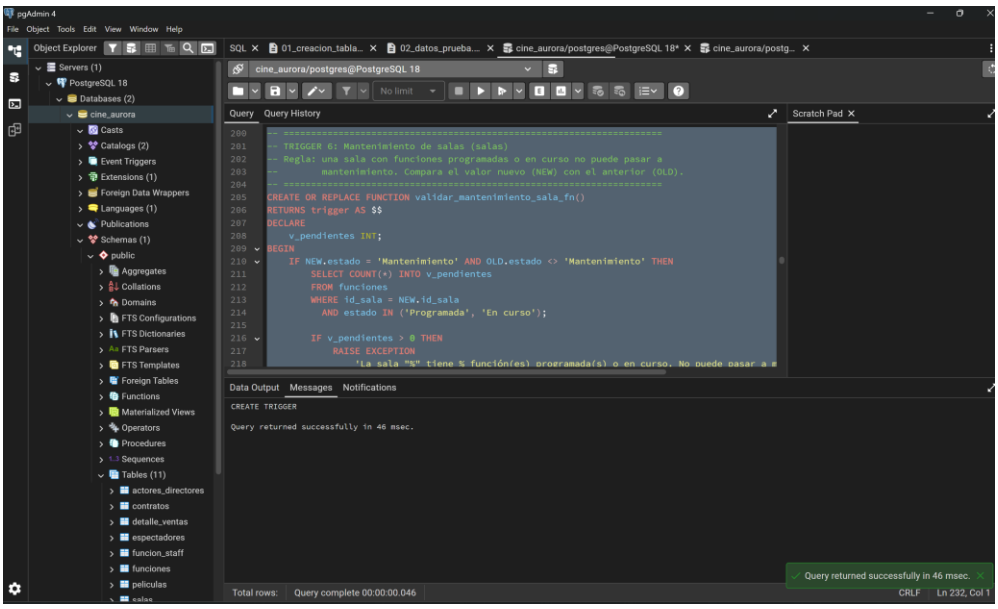


Trigger 6: trg_validar_mantenimiento_sala

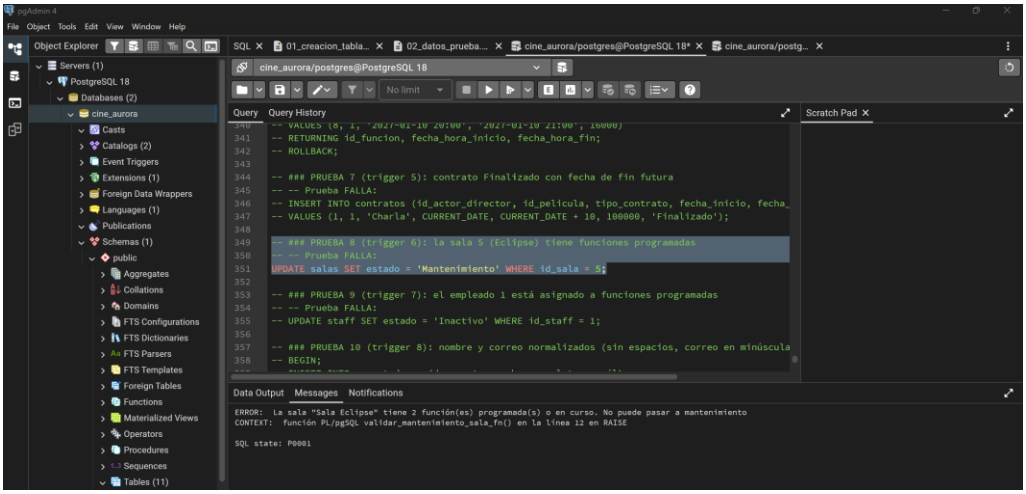
Tabla sobre la que actúa	salas
Evento	BEFORE UPDATE
Función asociada	validar_mantenimiento_sala_fn()
Regla de negocio que protege	Una sala no puede pasar a Mantenimiento si todavía tiene funciones programadas o en curso.

Usa OLD y NEW para revisar solo cuando el estado cambia a Mantenimiento. Cuenta las funciones Programadas o En curso de esa sala y, si hay alguna, lanza un RAISE EXCEPTION. Así ninguna función se queda sin sala.

Captura 14. Ejecución del trigger 6 (trg_validar_mantenimiento_sala)



Captura 15. Prueba del trigger 6 (trg_validar_mantenimiento_sala)

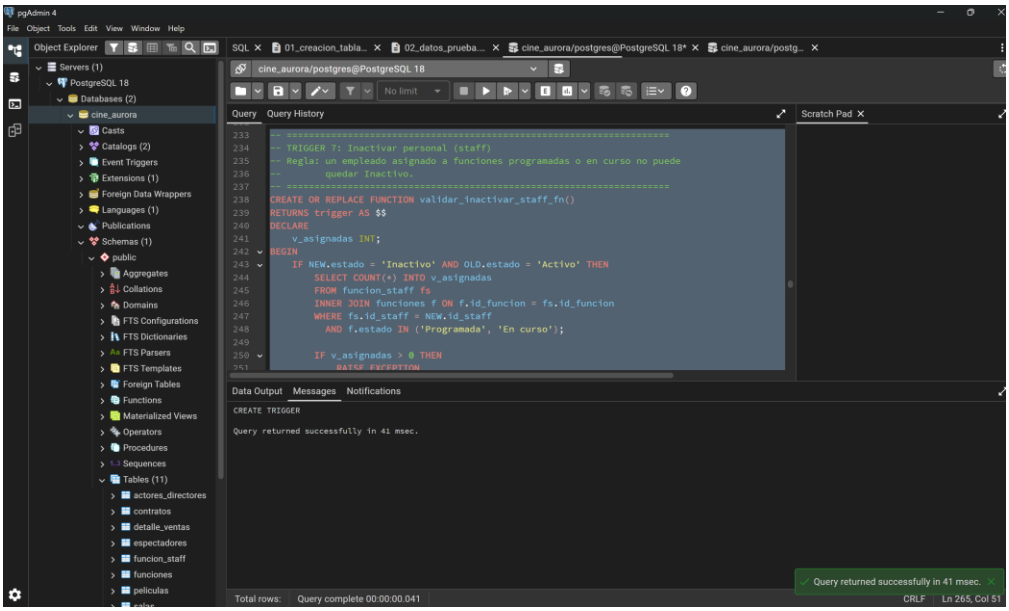


Trigger 7: trg_validar_inactivar_staff

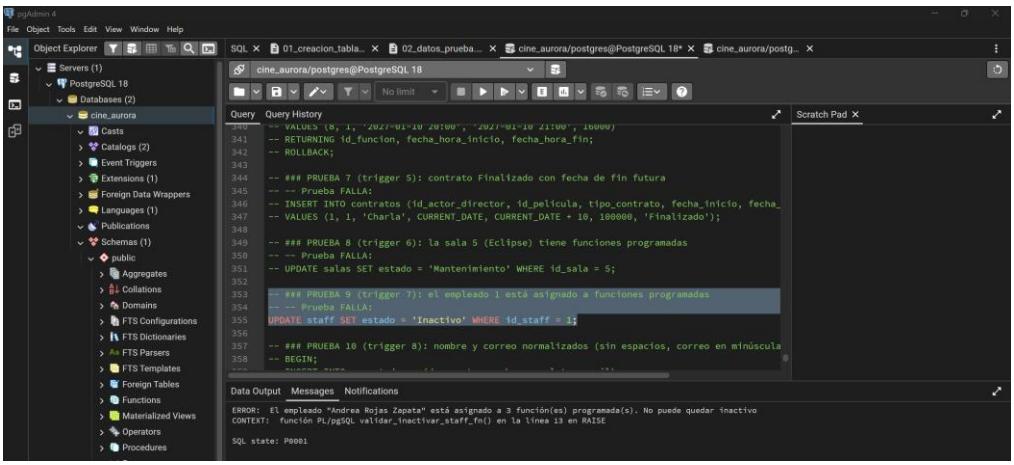
Tabla sobre la que actúa	staff
Evento	BEFORE UPDATE
Función asociada	validar_inactivar_staff_fn()
Regla de negocio que protege	No se puede inactivar a un empleado que está asignado a funciones programadas o en curso.

Cuando un empleado pasa de Activo a Inactivo, cuenta en cuántas funciones pendientes está asignado (tabla funcion_staff). Si tiene alguna, lanza un RAISE EXCEPTION para que esa función no se quede sin personal.

Captura 16. Ejecución del trigger 7 (trg_validar_inactivar_staff)



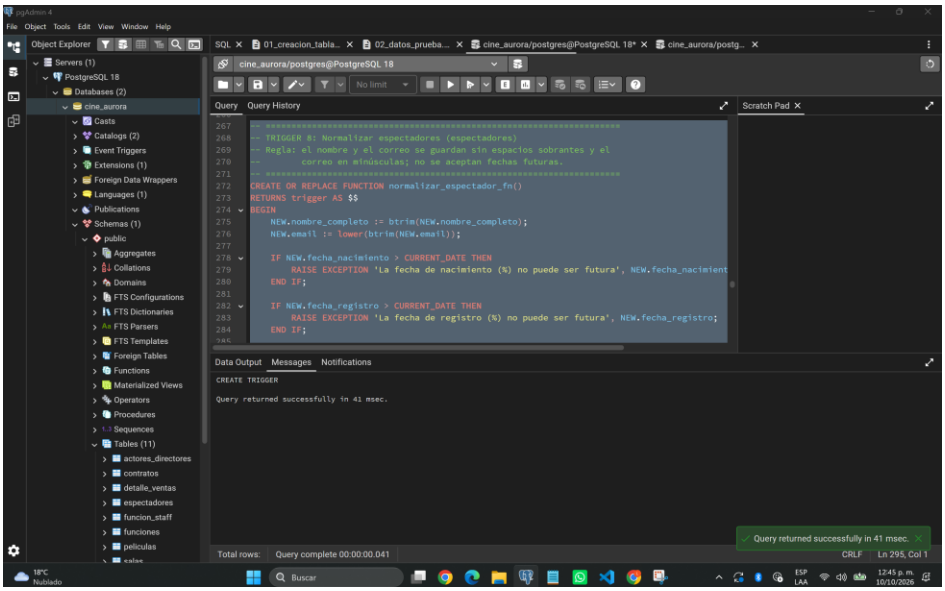
Captura 17. Prueba del trigger 7 (trg_validar_inactivar_staff)



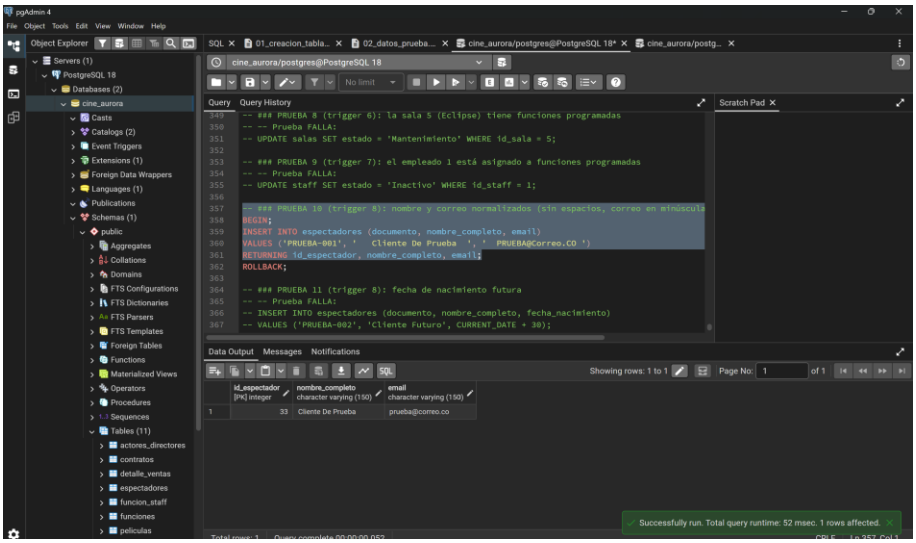
Trigger 8: trg_normalizar_espectador

Tabla sobre la que actúa	espectadores
Evento	BEFORE INSERT OR UPDATE
Función asociada	normalizar_espectador_fn()
Regla de negocio que protege	Los datos del espectador deben quedar limpios y sin fechas futuras.

Captura 18. Ejecución del trigger 8 (trg_normalizar_espectador)



Captura 19. Prueba del trigger 8 (trg_normalizar_espectador)

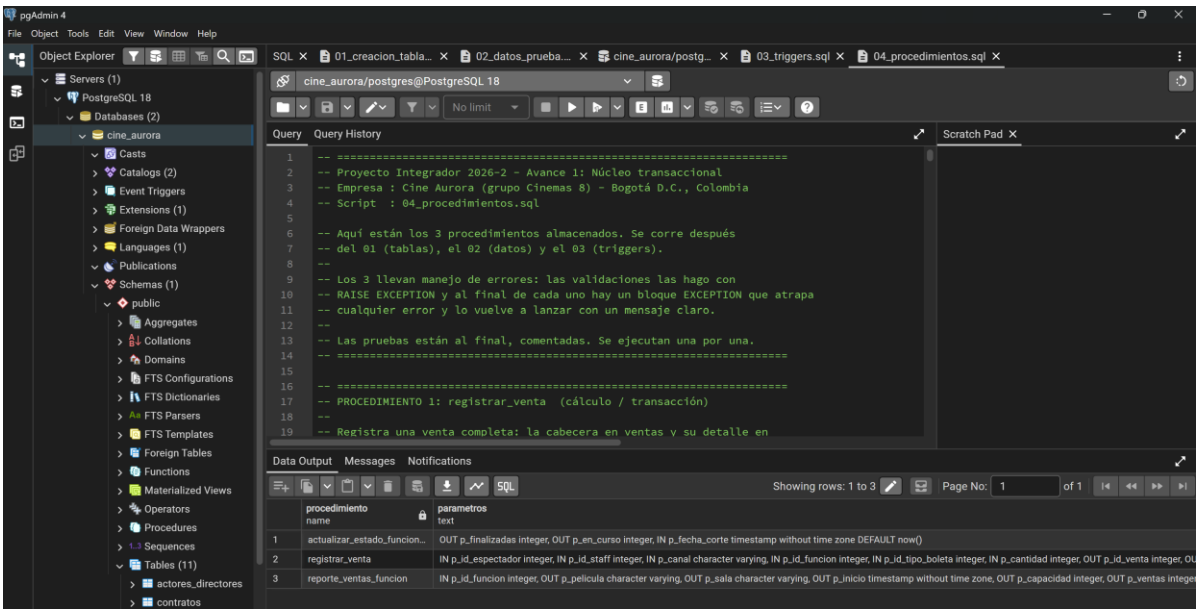


7. Procedimientos almacenados

El archivo 04_procedimientos.sql tiene 3 procedimientos, uno de cada tipo que piden: uno de cálculo o transacción, uno de reporte con agregación y uno de mantenimiento. Los tres validan con RAISE EXCEPTION y tienen un bloque EXCEPTION que atrapa cualquier error y lo vuelve a lanzar con un mensaje.

	Procedimiento	Tipo
1	registrar_venta	Cálculo / transacción
2	reporte_ventas_funcion	Reporte con agregación
3	actualizar_estado_funciones	Mantenimiento / operación

Captura 20. Ejecución del script 04_procedimientos.sql

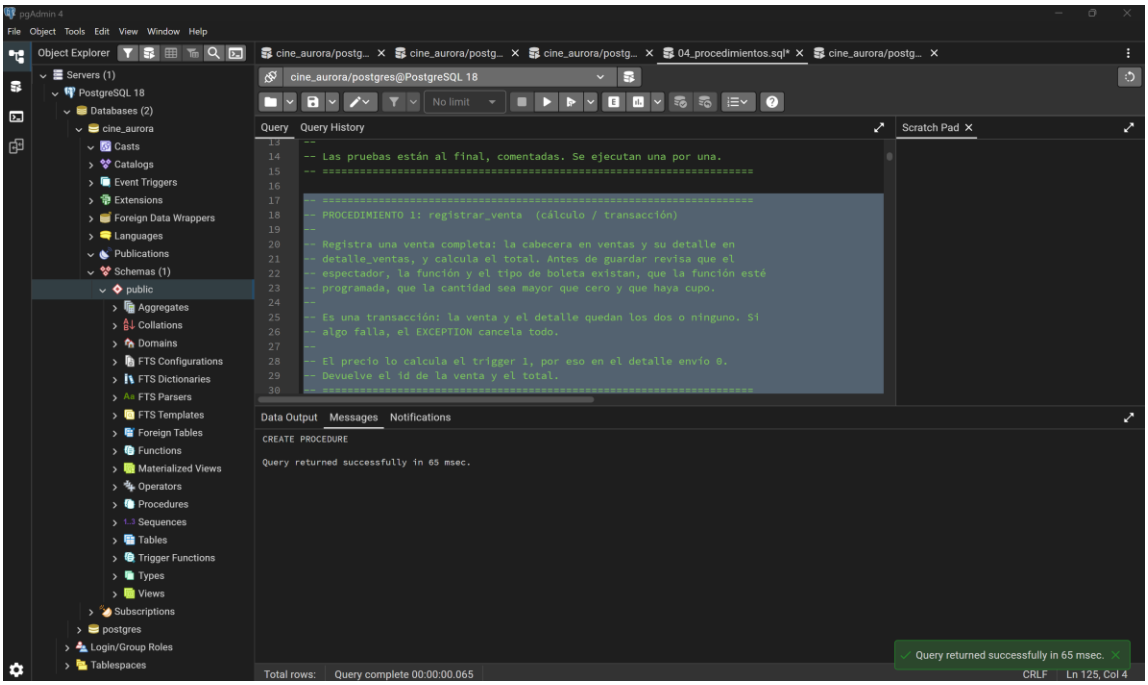


Procedimiento 1: registrar_venta

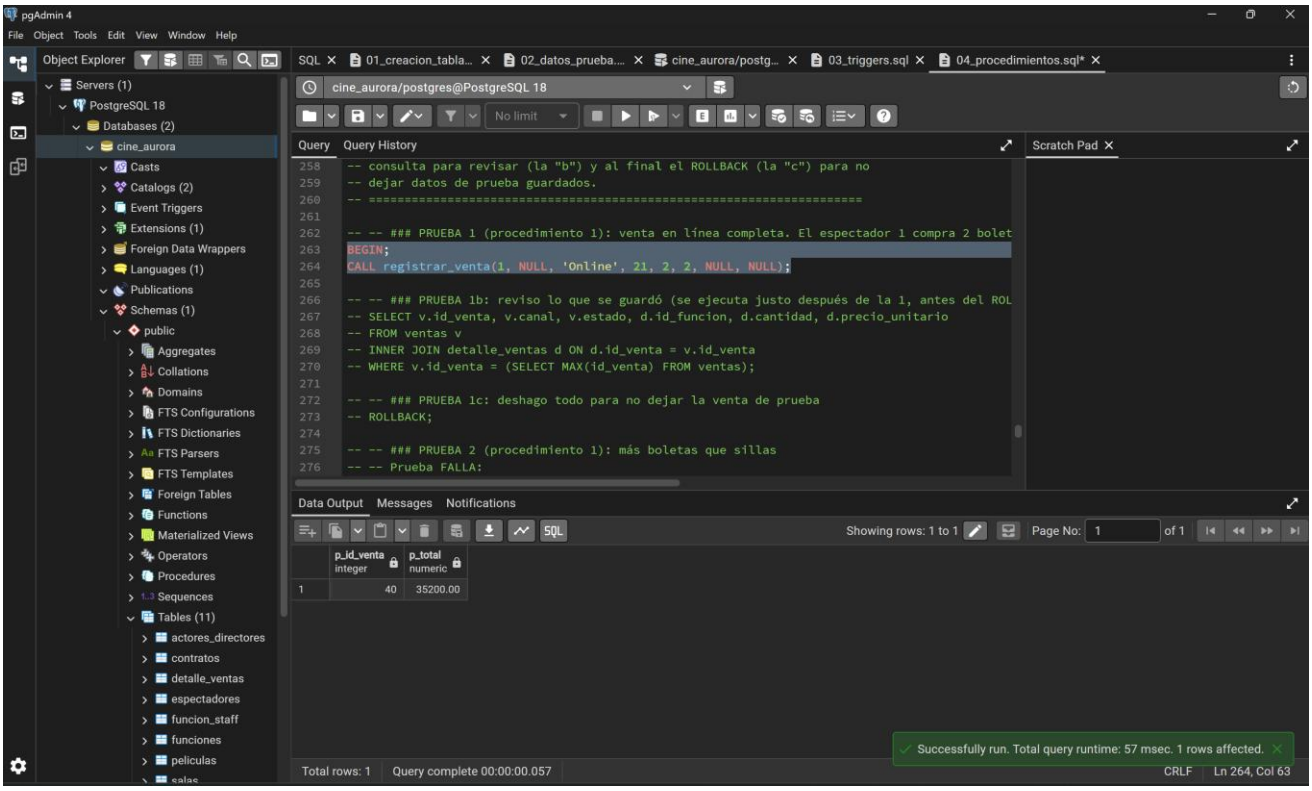
Tipo	Cálculo / transacción
Parámetros	Entran p_id_espectador, p_id_staff, p_canal, p_id_funcion, p_id_tipo_boleta y p_cantidad. Salen p_id_venta y p_total.
Propósito de negocio	Registra una venta completa: guarda la venta en ventas y su línea en detalle_ventas, y calcula el total (precio unitario por cantidad). Es una transacción, así que se guardan las dos cosas o no se guarda ninguna.

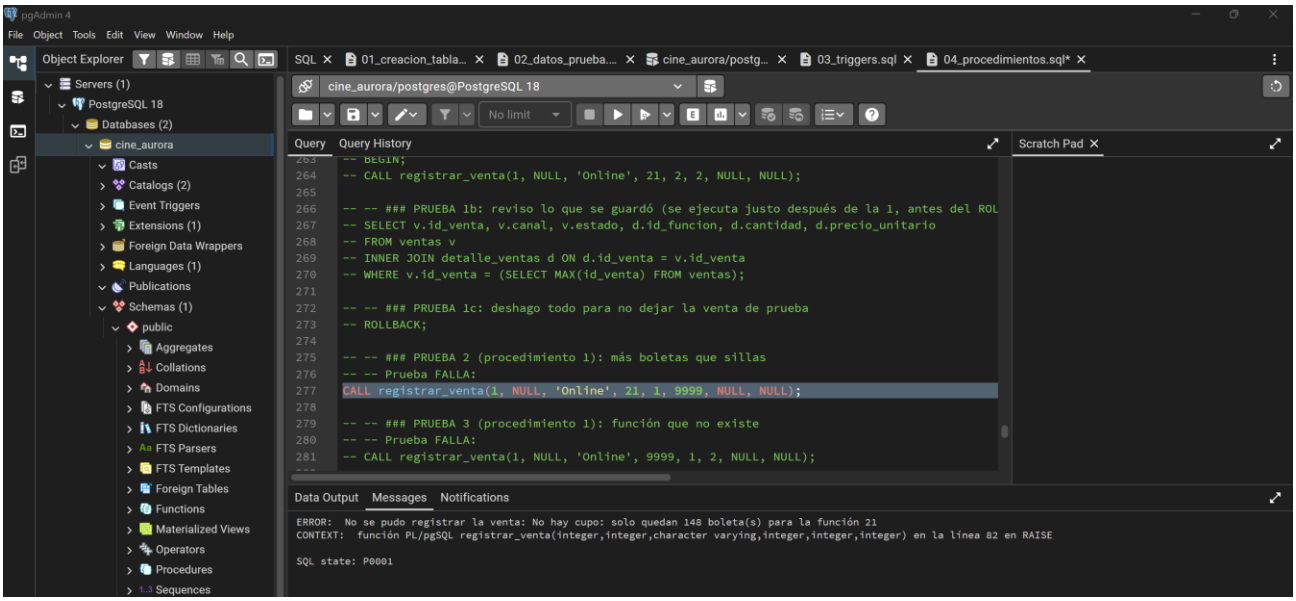
Validaciones	Que la cantidad sea mayor que cero, que el canal sea válido, que existan el espectador, el tipo de boleta y la función, que la función esté Programada y que haya cupo. Además, actúan los triggers 1 (precio), 2 (capacidad) y 3 (taquillero).
Manejo de excepciones / agregación	Cada validación usa RAISE EXCEPTION. El bloque EXCEPTION WHEN OTHERS cancela todo y lanza el error con el mensaje "No se pudo registrar la venta" seguido del motivo.

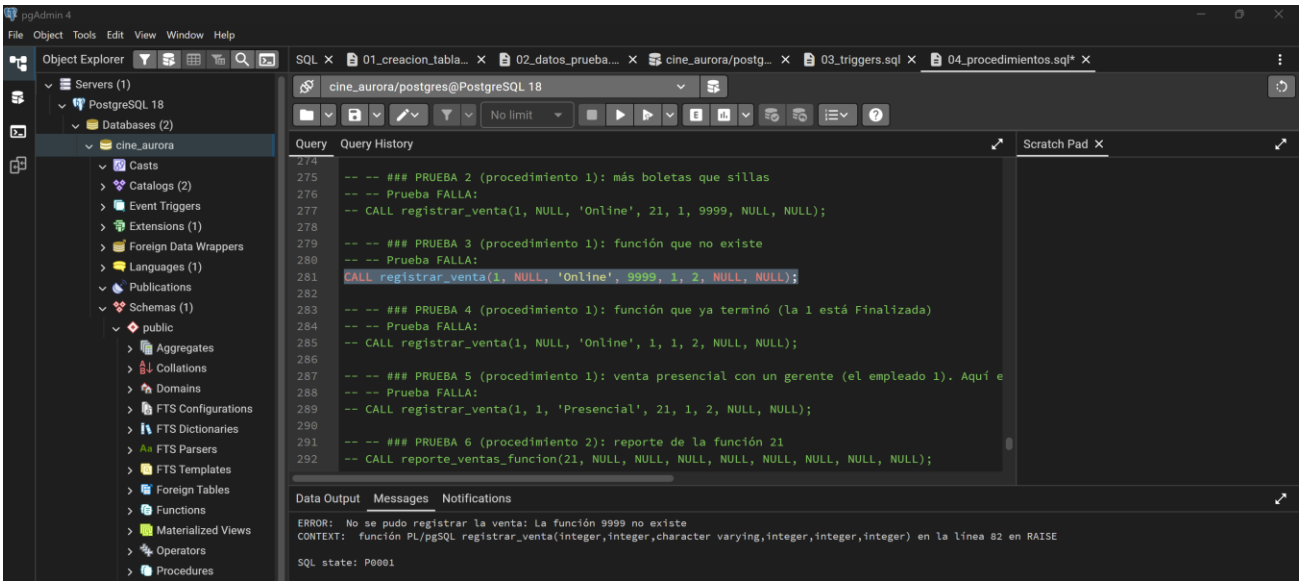
Captura 21. Ejecución del procedimiento 1 (registrar_venta)

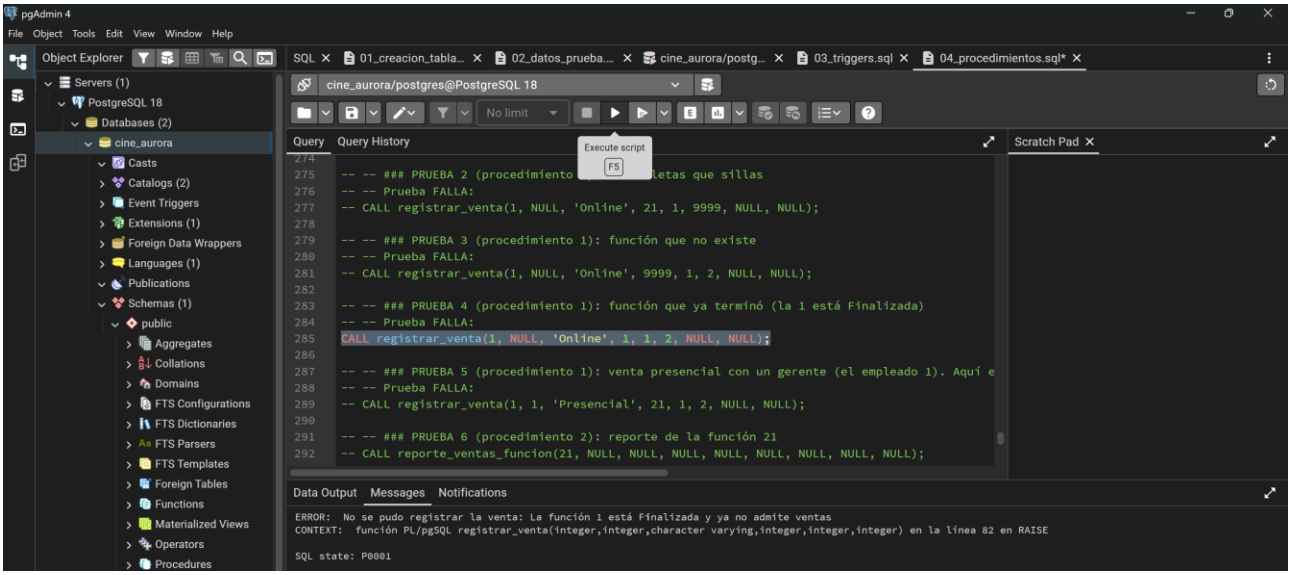


Captura 22. Pruebas del procedimiento 1 (registrar_venta)





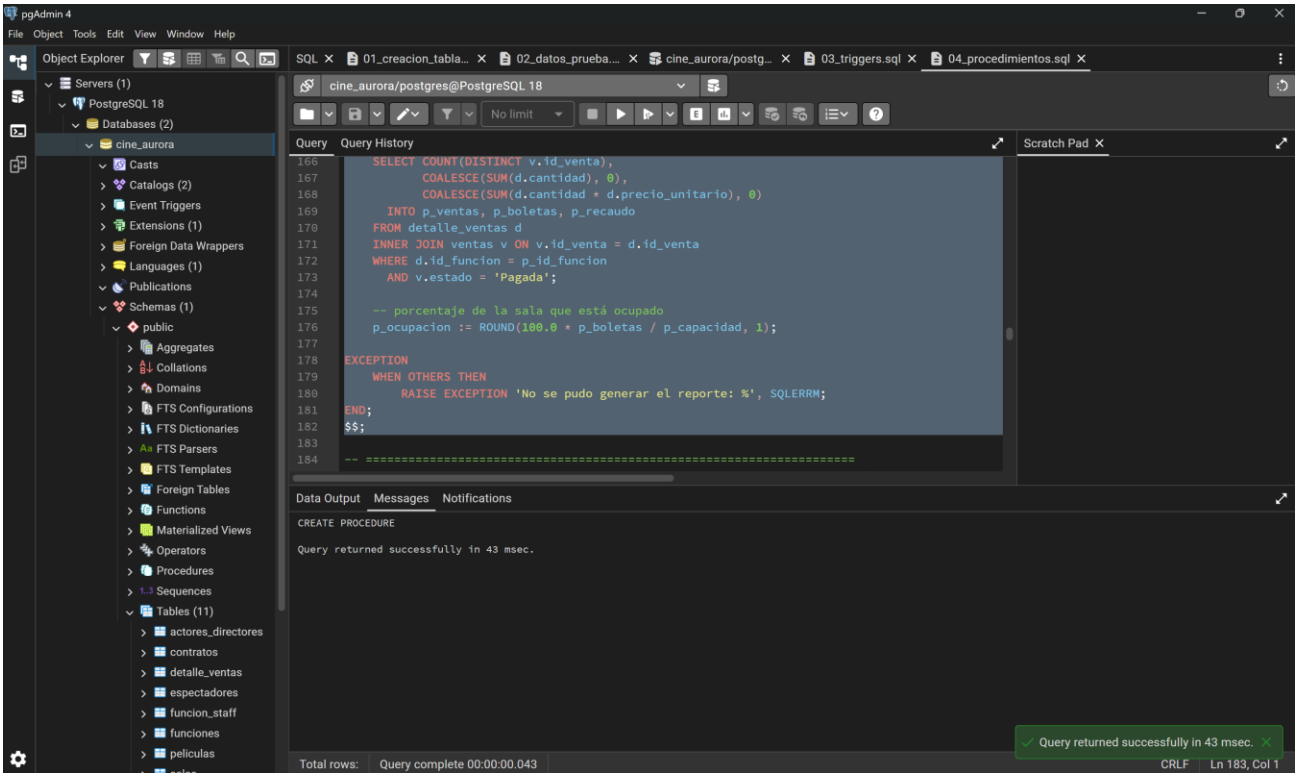

pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
Servers (1)
PostgreSQL 18
Databases (2)
cine_aurora
Casts
Catalogs (2)
Event Triggers
Extensions (1)
Foreign Data Wrappers
Languages (1)
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
SQL
01_creacion_tabla...
02_datos_prueba....
cine_aurora/postg...
03_triggers.sql
04_procedimientos.sql*
cine_aurora/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
-- -- ### PRUEBA 2 (procedimiento 1): más boletas que sillas
-- -- Prueba FALLA:
-- CALL registrar_venta(1, NULL, 'Online', 21, 1, 9999, NULL, NULL);
-- -- ### PRUEBA 3 (procedimiento 1): función que no existe
-- -- Prueba FALLA:
CALL registrar_venta(1, NULL, 'Online', 9999, 1, 2, NULL, NULL);
-- -- ### PRUEBA 4 (procedimiento 1): función que ya terminó (la 1 está Finalizada)
-- -- Prueba FALLA:
-- CALL registrar_venta(1, NULL, 'Online', 1, 1, 2, NULL, NULL);
-- -- ### PRUEBA 5 (procedimiento 1): venta presencial con un gerente (el empleado 1). Aquí e
-- -- Prueba FALLA:
-- CALL registrar_venta(1, 1, 'Presencial', 21, 1, 2, NULL, NULL);
-- -- ### PRUEBA 6 (procedimiento 2): reporte de la función 21
-- CALL reporte_ventas_funcion(21, NULL, NULL, NULL, NULL, NULL, NULL, NULL);
Data Output
Messages
Notifications
ERROR: No se pudo registrar la venta: La función 9999 no existe
CONTEXT: función PL/pgSQL registrar_venta(integer,integer,character varying,integer,integer,integer) en la línea 82 en RAISE
SQL state: P0001


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
Servers (1)
PostgreSQL 18
Databases (2)
cine_aurora
Casts
Catalogs (2)
Event Triggers
Extensions (1)
Foreign Data Wrappers
Languages (1)
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
SQL
01_creacion_tabla...
02_datos_prueba....
cine_aurora/postg...
03_triggers.sql
04_procedimientos.sql*
cine_aurora/postgres@PostgreSQL 18
No limit
Execute script
F5
Query
Query History
Scratch Pad
-- -- ### PRUEBA 2 (procedimiento
letas que sillas
-- -- Prueba FALLA:
-- CALL registrar_venta(1, NULL, 'Online', 21, 1, 9999, NULL, NULL);
-- -- ### PRUEBA 3 (procedimiento 1): función que no existe
-- -- Prueba FALLA:
-- CALL registrar_venta(1, NULL, 'Online', 9999, 1, 2, NULL, NULL);
-- -- ### PRUEBA 4 (procedimiento 1): función que ya terminó (la 1 está Finalizada)
-- -- Prueba FALLA:
CALL registrar_venta(1, NULL, 'Online', 1, 1, 2, NULL, NULL);
-- -- ### PRUEBA 5 (procedimiento 1): venta presencial con un gerente (el empleado 1). Aquí e
-- -- Prueba FALLA:
-- CALL registrar_venta(1, 1, 'Presencial', 21, 1, 2, NULL, NULL);
-- -- ### PRUEBA 6 (procedimiento 2): reporte de la función 21
-- CALL reporte_ventas_funcion(21, NULL, NULL, NULL, NULL, NULL, NULL, NULL);
Data Output
Messages
Notifications
ERROR: No se pudo registrar la venta: La función 1 está Finalizada y ya no admite ventas
CONTEXT: función PL/pgSQL registrar_venta(integer,integer,character varying,integer,integer,integer) en la línea 82 en RAISE
SQL state: P0001

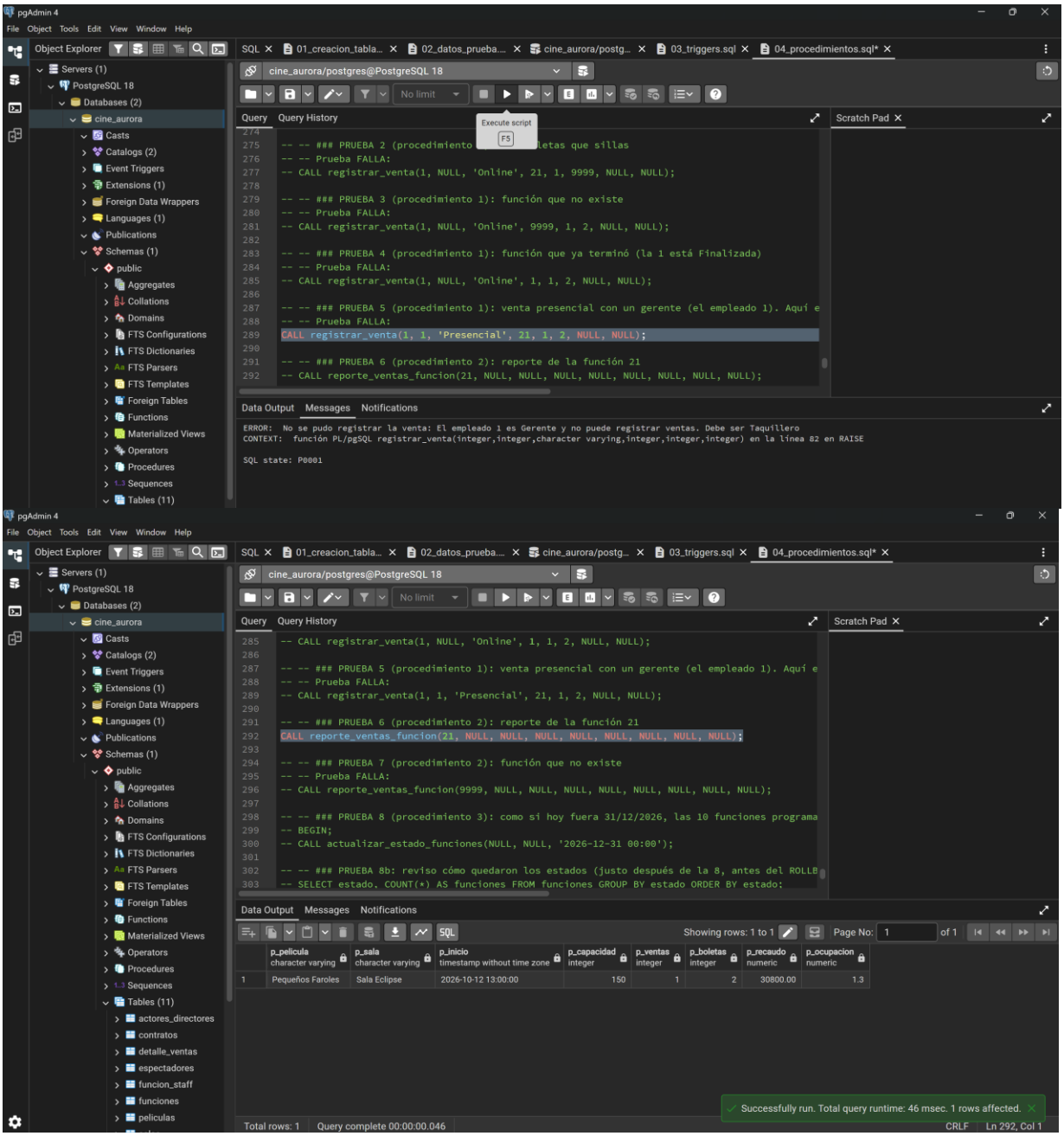
Procedimiento 2: reporte_ventas_funcion

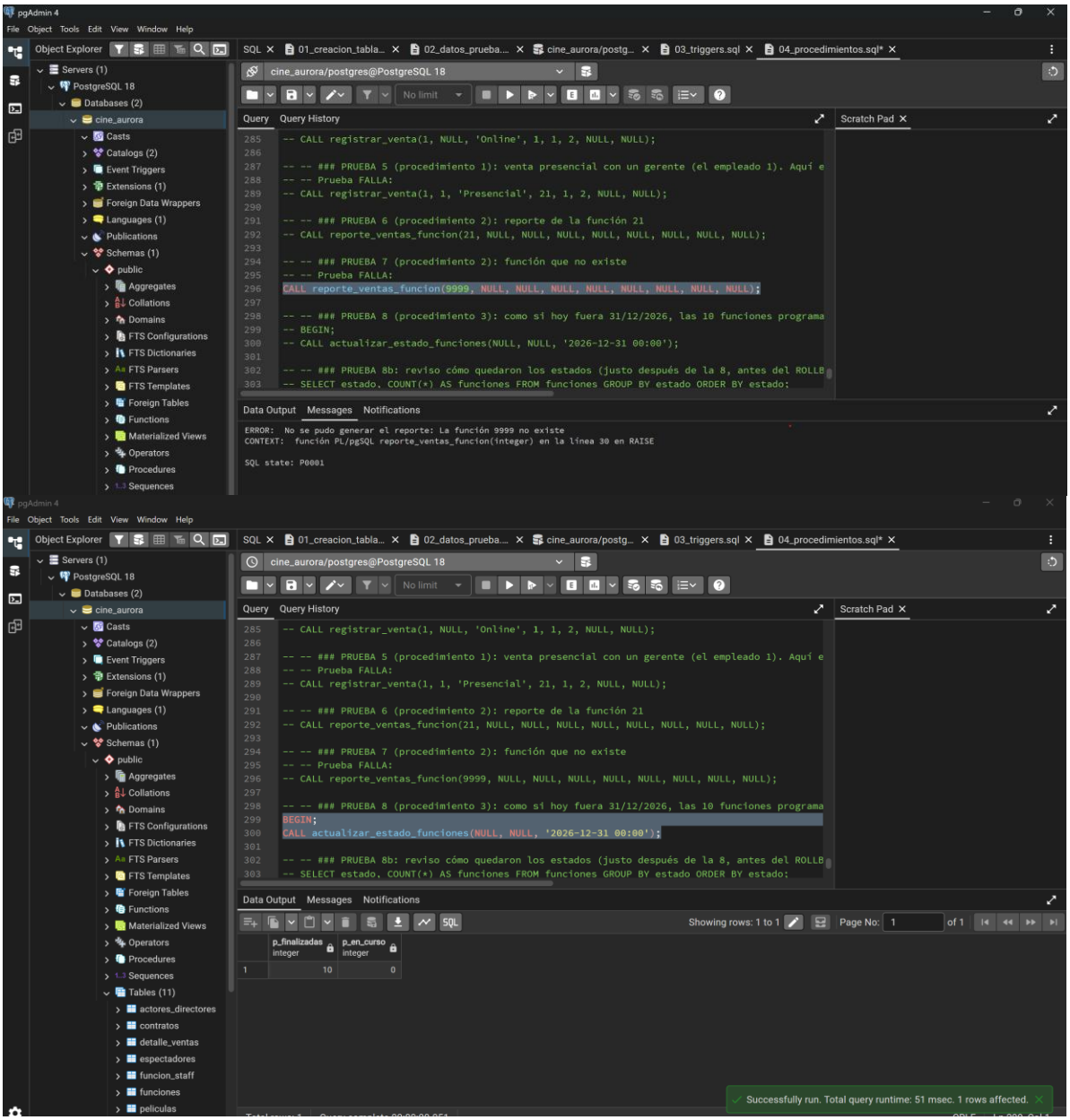
Tipo	Reporte con agregación
Parámetros	Entra p_id_funcion. Salen p_pelicula, p_sala, p_inicio, p_capacidad, p_ventas, p_boletas, p_recaudo y p_ocupacion.
Propósito de negocio	Hace el reporte de una función: la película, la sala, la hora de inicio, cuántas ventas tiene, cuántas boletas se vendieron, cuánto se recaudó y qué porcentaje de la sala está ocupado.
Validaciones	Que la función exista. Solo se cuentan las ventas Pagadas, las anuladas no suman.
Manejo de excepciones / agregación	Usa COUNT (DISTINCT id_venta), SUM (cantidad) y SUM (cantidad * precio_unitario). Si hay un error, el bloque EXCEPTION lo lanza otra vez con el mensaje "No se pudo generar el reporte" seguido del motivo.

Captura 23. Ejecución del procedimiento 2 (reporte_ventas_funcion)



Captura 24. Pruebas del procedimiento 2 (reporte_ventas_funcion)

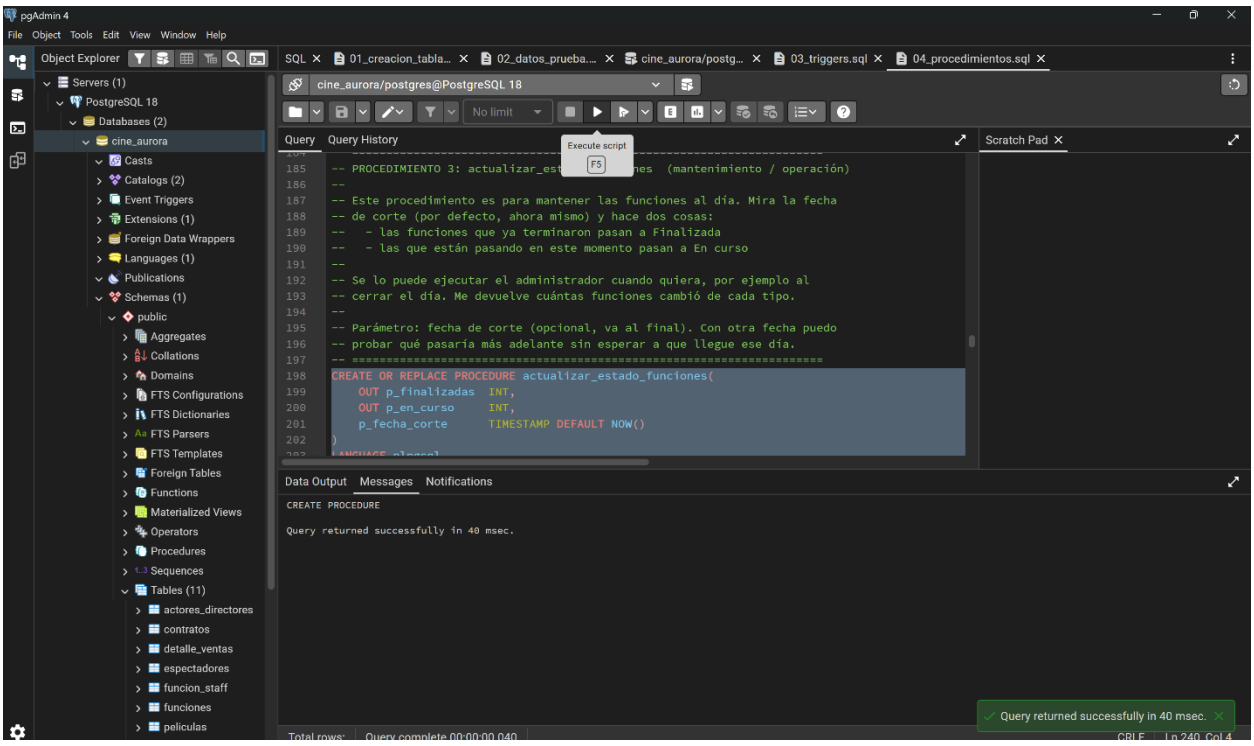



pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
Servers (1)
PostgreSQL 18
Databases (2)
cine_aurora
Casts
Catalogs (2)
Event Triggers
Extensions (1)
Foreign Data Wrappers
Languages (1)
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
SQL
01_creacion_tabla...
02_datos_prueba...
cine_aurora/postg...
03_triggers.sql
04_procedimientos.sql*
cine_aurora/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
285 -- CALL registrar_venta(1, NULL, 'Online', 1, 1, 2, NULL, NULL);
286
287 -- -- ### PRUEBA 5 (procedimiento 1): venta presencial con un gerente (el empleado 1). Aquí e
288 -- -- Prueba FALLA:
289 -- CALL registrar_venta(1, 1, 'Presencial', 21, 1, 2, NULL, NULL);
290
291 -- -- ### PRUEBA 6 (procedimiento 2): reporte de la función 21
292 -- CALL reporte_ventas_funcion(21, NULL, NULL, NULL, NULL, NULL, NULL, NULL);
293
294 -- -- ### PRUEBA 7 (procedimiento 2): función que no existe
295 -- -- Prueba FALLA:
296 CALL reporte_ventas_funcion(9999, NULL, NULL, NULL, NULL, NULL, NULL, NULL);
297
298 -- -- ### PRUEBA 8 (procedimiento 3): como si hoy fuera 31/12/2026, las 10 funciones programa
299 -- BEGIN;
300 -- CALL actualizar_estado_funciones(NULL, NULL, '2026-12-31 00:00');
301
302 -- -- ### PRUEBA 8b: reviso cómo quedaron los estados (justo después de la 8, antes del ROLLB
303 -- SELECT estado, COUNT(*) AS funciones FROM funciones GROUP BY estado ORDER BY estado;
Data Output
Messages
Notifications
ERROR: No se pudo generar el reporte: La función 9999 no existe
CONTEXT: función PL/pgSQL reporte_ventas_funcion(integer) en la línea 30 en RAISE
SQL state: P0001
pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
Servers (1)
PostgreSQL 18
Databases (2)
cine_aurora
Casts
Catalogs (2)
Event Triggers
Extensions (1)
Foreign Data Wrappers
Languages (1)
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (11)
actores_directores
contratos
detalle_ventas
espectadores
funcion_staff
funciones
peliculas
SQL
01_creacion_tabla...
02_datos_prueba...
cine_aurora/postg...
03_triggers.sql
04_procedimientos.sql*
cine_aurora/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
285 -- CALL registrar_venta(1, NULL, 'Online', 1, 1, 2, NULL, NULL);
286
287 -- -- ### PRUEBA 5 (procedimiento 1): venta presencial con un gerente (el empleado 1). Aquí e
288 -- -- Prueba FALLA:
289 -- CALL registrar_venta(1, 1, 'Presencial', 21, 1, 2, NULL, NULL);
290
291 -- -- ### PRUEBA 6 (procedimiento 2): reporte de la función 21
292 -- CALL reporte_ventas_funcion(21, NULL, NULL, NULL, NULL, NULL, NULL, NULL);
293
294 -- -- ### PRUEBA 7 (procedimiento 2): función que no existe
295 -- -- Prueba FALLA:
296 -- CALL reporte_ventas_funcion(9999, NULL, NULL, NULL, NULL, NULL, NULL, NULL);
297
298 -- -- ### PRUEBA 8 (procedimiento 3): como si hoy fuera 31/12/2026, las 10 funciones programa
299 BEGIN;
300 CALL actualizar_estado_funciones(NULL, NULL, '2026-12-31 00:00');
301
302 -- -- ### PRUEBA 8b: reviso cómo quedaron los estados (justo después de la 8, antes del ROLLB
303 -- SELECT estado, COUNT(*) AS funciones FROM funciones GROUP BY estado ORDER BY estado;
Data Output
Messages
Notifications
Showing rows: 1 to 1
Page No: 1 of 1
p_finalizadas integer
p_en_curso integer
1 10 0
Successfully run. Total query runtime: 51 msec. 1 rows affected.

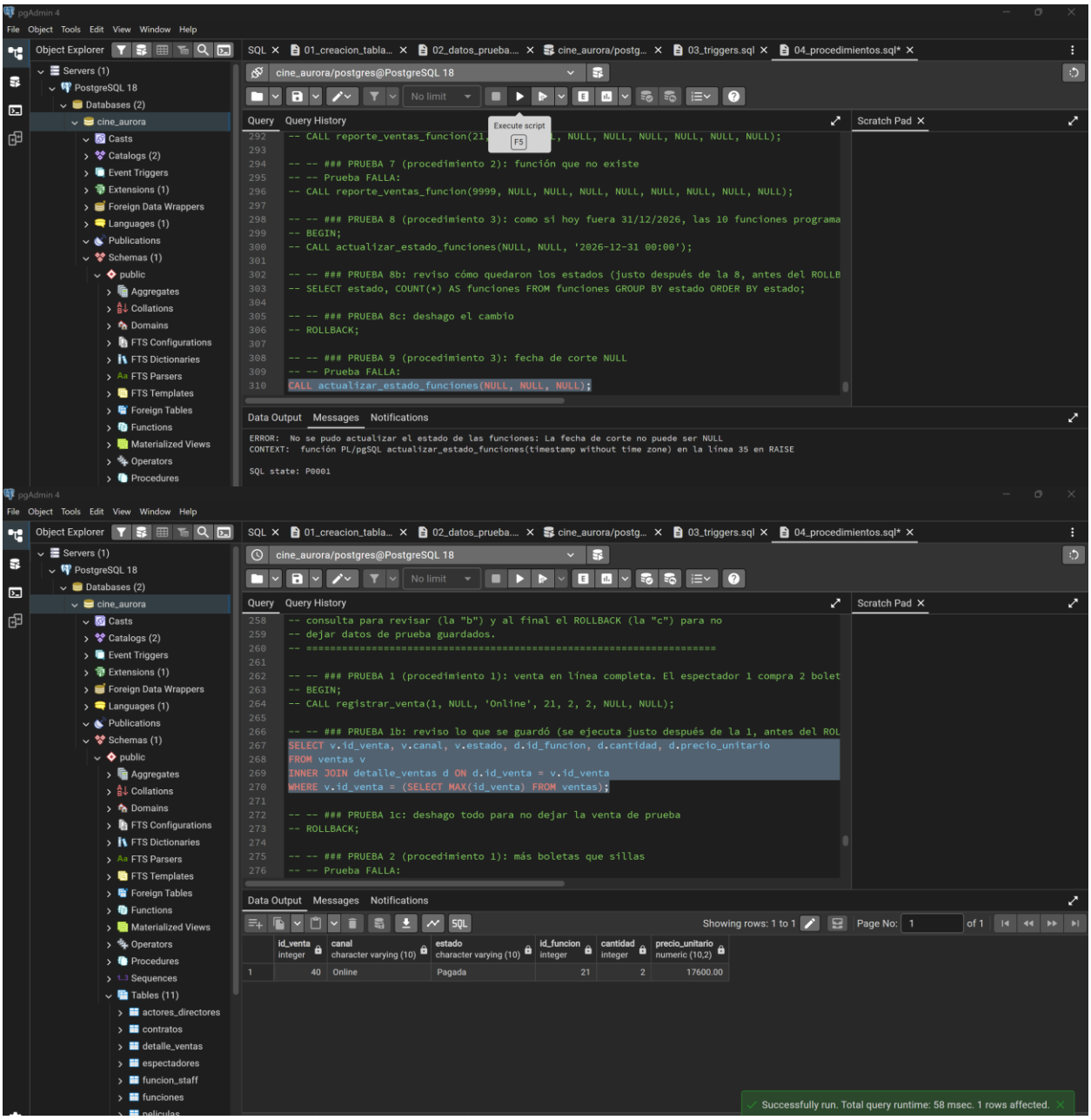
Procedimiento 3: actualizar_estado_funciones

Tipo	Mantenimiento / operación
Parámetros	Entra p_fecha_corte (es opcional y por defecto es NOW ()). Salen p_finalizadas y p_en_curso.
Propósito de negocio	Mantiene al día el estado de las funciones según una fecha de corte: las que ya terminaron pasan a Finalizada y las que están pasando en ese momento pasan a En curso. Devuelve cuántas cambió de cada tipo. Si se usa otra fecha de corte se puede probar qué pasaría más adelante.
Validaciones	Que la fecha de corte no sea NULL.
Manejo de excepciones / agregación	Lanza un RAISE EXCEPTION si la fecha es nula, y el bloque EXCEPTION vuelve a lanzar cualquier error con el mensaje "No se pudo actualizar el estado de las funciones" seguido del motivo.

Captura 25. Ejecución del procedimiento 3 (actualizar_estado_funciones)



Captura 26. Pruebas del procedimiento 3 (actualizar_estado_funciones)



8. Consultas SQL

El archivo 05_consultas.sql tiene 3 consultas que responden preguntas que se haría la administración del cine. Las tres usan JOIN, funciones de agregación y subconsultas.

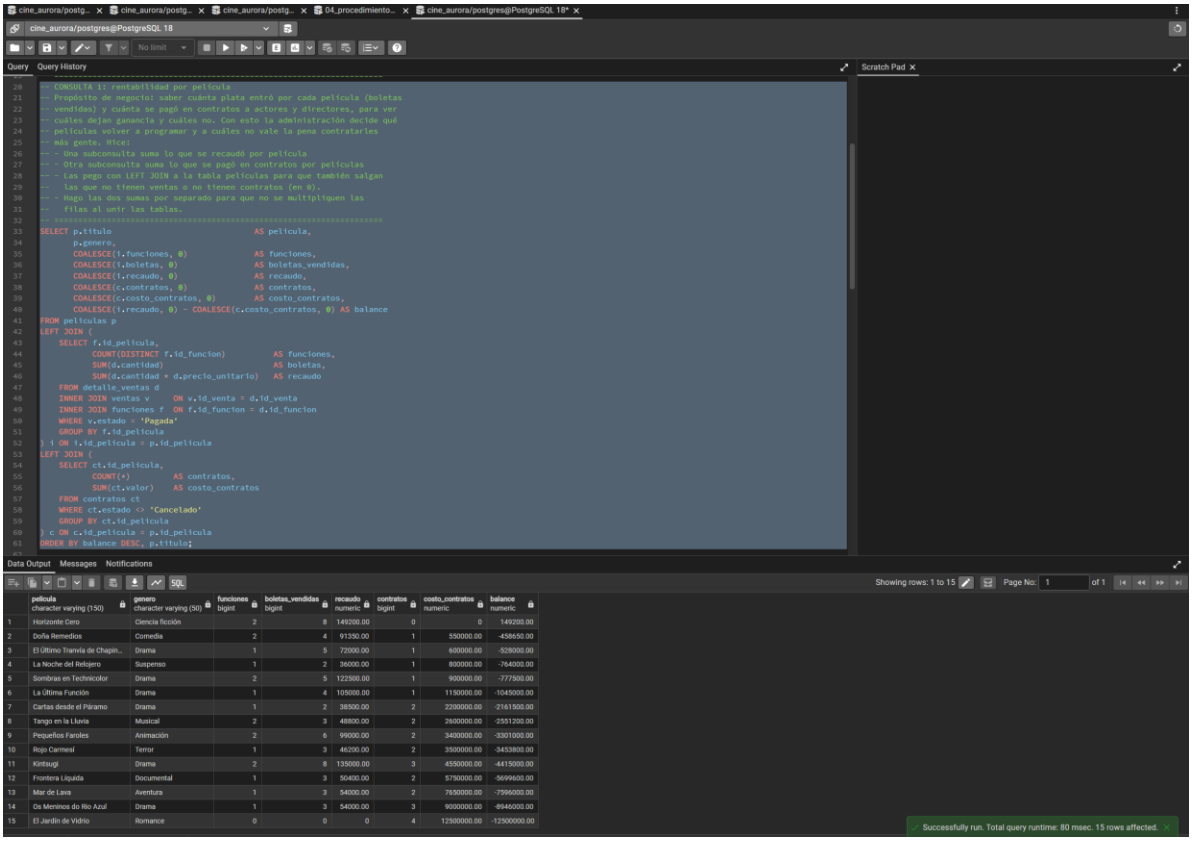
Consulta 1: Rentabilidad por película

Pregunta de negocio	¿Qué películas dejan más dinero y cuáles no pagan lo que cuestan?
Propósito de negocio	Compara lo que se recaudó con las boletas de cada película contra lo que se pagó en contratos a actores y directores. Sirve para decidir qué películas conviene volver a programar y con cuáles no vale la pena seguir contratando artistas.

Cómo está construida:

- Una subconsulta en el FROM suma el recaudo por película (cantidad por precio_unitario), solo de ventas Pagadas.
- Otra subconsulta suma el valor de los contratos que no están cancelados, por película.
- Se unen con LEFT JOIN a películas para que también salgan las películas sin ventas o sin contratos (con 0).
- Las dos sumas se hacen por separado para que las filas no se multipliquen al unir las tablas.
- El balance es el recaudo menos el costo de los contratos, ordenado de mayor a menor.

Captura 27. Consulta 1, código y resultado



```
-- ============================================================
-- CONSULTA 1: rentabilidad por película
-- Propósito de negocio: saber cuánta plata entró por cada película (boletas
-- vendidas) y cuánta se pagó en contratos a actores y directores, para ver
-- cuáles dejan ganancia y cuáles no. Con esto la administración decide qué
-- películas volver a programar y a cuáles no vale la pena contratarles
-- más gente. Hice:
-- - Una subconsulta suma lo que se recaudó por película
-- - Otra subconsulta suma lo que se pagó en contratos por películas
-- - Las pego con LEFT JOIN a la tabla películas para que también salgan
--   las que no tienen ventas o no tienen contratos (en 0).
-- - Hago las dos sumas por separado para que no se multipliquen las
--   filas al unir las tablas.
-- ============================================================
SELECT p.titulo                          AS película,
       p.genero,
       COALESCE(f.funciones, 0)          AS funciones,
       COALESCE(f.boletas, 0)            AS boletas_vendidas,
       COALESCE(f.recaudo, 0)            AS recaudo,
       COALESCE(c.contratos, 0)          AS contratos,
       COALESCE(c.costo_contratos, 0)    AS costo_contratos,
       COALESCE(f.recaudo, 0) - COALESCE(c.costo_contratos, 0) AS balance
FROM películas p
LEFT JOIN (
    SELECT f.id_película,
           COUNT(DISTINCT f.id_función)      AS funciones,
           SUM(d.cantidad)                   AS boletas,
           SUM(d.cantidad * d.precio_unitario)  AS recaudo
    FROM detalle_ventas d
    INNER JOIN ventas v     ON v.id_venta = d.id_venta
    INNER JOIN funciones f  ON f.id_función = d.id_función
    WHERE v.estado = 'Pagada'
    GROUP BY f.id_película
) f ON f.id_película = p.id_película
LEFT JOIN (
    SELECT ct.id_película,
           COUNT(*)          AS contratos,
           SUM(ct.valor)     AS costo_contratos
    FROM contratos ct
    WHERE ct.estado <> 'Cancelado'
    GROUP BY ct.id_película
) c ON c.id_película = p.id_película
ORDER BY balance DESC, p.título;
```

	película	genero	funciones	boletas_vendidas	recaudo	contratos	costo_contratos	balance
1	Horizonte Cero	Ciencia ficción	2	8	149200.00	0	0	149200.00
2	Doña Remedios	Comedia	2	4	91350.00	1	550000.00	-458650.00
3	El Último Tranvía de Chapin...	Drama	1	5	72000.00	1	600000.00	-528000.00
4	La Noche del Relojero	Suspenso	1	2	36000.00	1	800000.00	-764000.00
5	Sombras en Technicolor	Drama	2	5	122500.00	1	900000.00	-777500.00
6	La Última Función	Drama	1	4	105000.00	1	1150000.00	-1045000.00
7	Cartas desde el Páramo	Drama	1	2	38500.00	2	2200000.00	-2161500.00
8	Tango en la Lluvia	Musical	2	3	48800.00	2	2600000.00	-2551200.00
9	Pequeños Faroles	Animación	2	6	99000.00	2	3400000.00	-3301000.00
10	Rojo Carmesí	Terror	1	3	46200.00	2	3500000.00	-3453800.00
11	Kintsugi	Drama	2	8	135000.00	3	4550000.00	-4415000.00
12	Frontera Líquida	Documental	1	3	50400.00	2	5750000.00	-5699600.00
13	Mar de Lava	Aventura	1	3	54000.00	2	7650000.00	-7596000.00
14	Os Meninos do Rio Azul	Drama	1	3	54000.00	3	9000000.00	-8946000.00
15	El Jardín de Vidrio	Romance	0	0	0	4	12500000.00	-12500000.00

Interpretación de los resultados: De las 15 películas, solo Horizonte Cero tiene balance positivo $149.200. Las otras 14 quedan en negativo porque lo que se pagó en contratos es mucho más que lo recaudado con las boletas. El caso más extremo es El Jardín de Vidrio, que no tiene funciones ni boletas vendidas, pero sí 4 contratos, por eso su balance es de -$12.500.000. Parte de esto se debe a que los datos de prueba tienen pocas ventas, pero igual permite ver qué películas no están pagando lo que cuestan. Con este resultado la administración podría dejar de contratar artistas para las películas con peor balance y programar más veces Horizonte Cero.

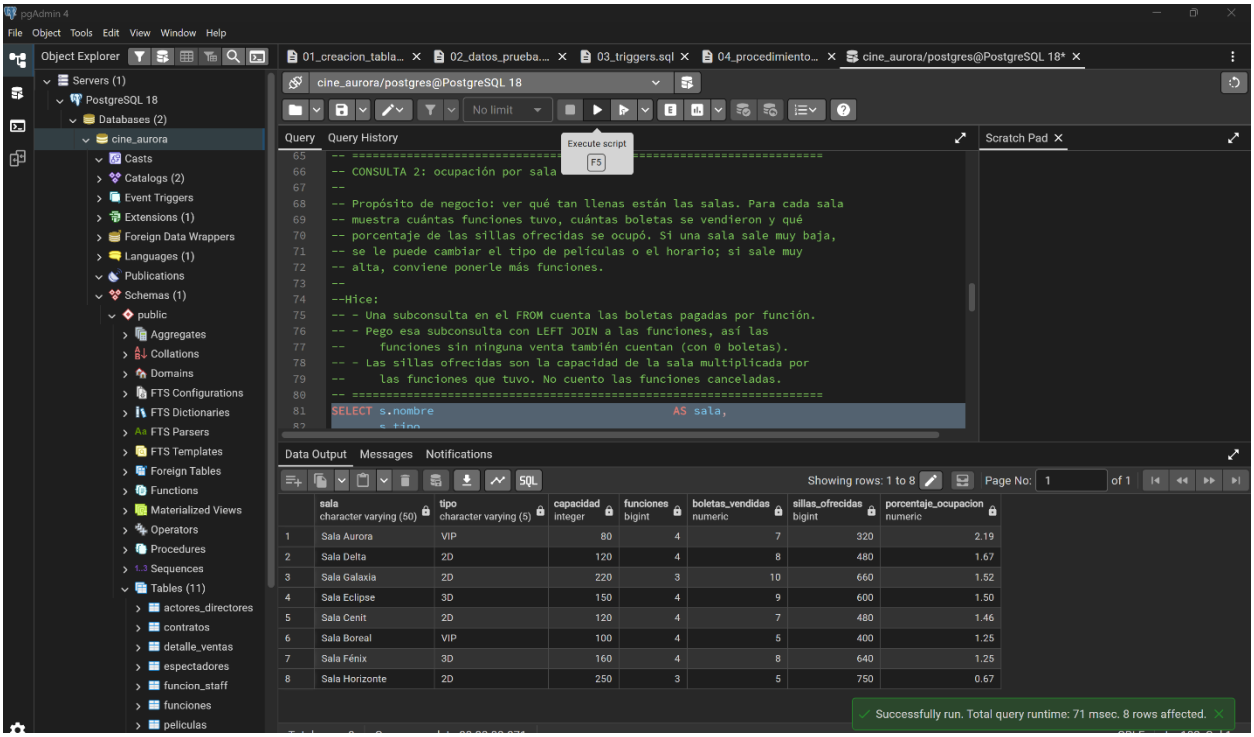
Consulta 2: Ocupación por sala

Pregunta de negocio	¿Qué salas se están llenando y cuáles quedan vacías?
Propósito de negocio	Muestra por sala cuántas funciones tuvo, cuántas boletas se vendieron y qué porcentaje de las sillas ofrecidas se ocupó. Si la ocupación es baja se puede cambiar el tipo de películas o el horario, y si es muy alta conviene ponerle más funciones.

Cómo está construida:

- Una subconsulta en el FROM cuenta las boletas pagadas por función.
- Se une con LEFT JOIN a funciones para que también cuenten las funciones sin ventas (0 boletas).
- Las sillas ofrecidas son la capacidad de la sala por el número de funciones, sin contar las canceladas.
- Se agrupa por sala y el porcentaje de ocupación se calcula con ROUND.

Captura 28. Consulta 2, código y resultado



Interpretación de los resultados: La consulta muestra la ocupación de las 8 salas y en todas es muy baja, ninguna llega al 2,2 % de las sillas ofrecidas. La Sala Aurora VIP, 80 sillas es la más ocupada con 2,19 % 7 boletas en 4 funciones, seguida de la Sala Delta con 1,67 % y la Sala Galaxia con 1,52 %. La menos ocupada es la Sala Horizonte 2D, 250 sillas con 0,67 %, porque es la más grande y solo vendió 5 boletas en 3 funciones. Con esto la administración vería que por ahora no conviene agregar más funciones, sino revisar los horarios y las películas, hacer promociones y programar las películas con menos público en salas más pequeñas. Los porcentajes salen bajos porque los datos de prueba tienen pocas ventas 59 boletas en total

Consulta 3: Espectadores que gastan más que el promedio

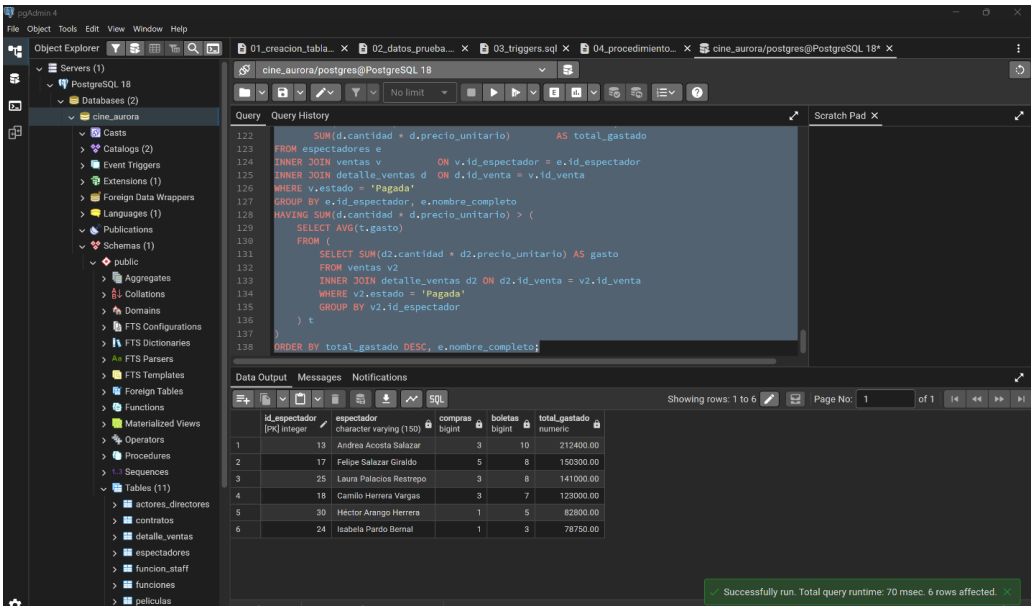
Pregunta de negocio	¿Quiénes son los clientes más valiosos?

Propósito de negocio	Busca a los espectadores que han gastado más que el promedio de los que compraron, para ofrecerles promociones o un programa de fidelidad.

Cómo está construida:

- Se unen espectadores, ventas y detalle_ventas, solo con ventas Pagadas.
- Se agrupa por espectador para sacar sus compras, sus boletas y el total gastado.
- El HAVING compara ese total con una subconsulta que calcula el promedio de gasto. Se usa HAVING y no WHERE porque se compara contra una suma.

Captura 29. Consulta 3, código y resultado



Interpretación de los resultados: La consulta devolvió 6 espectadores que gastaron más que el promedio de los clientes que compraron. Andrea Acosta Salazar es la que más gastó, $212.400 en 10 boletas, seguida de Felipe Salazar Giraldo $150.300, que además es quien más compras hizo 5, y de Laura Palacios Restrepo $141.000. Héctor Arango Herrera $82.800 e Isabela Pardo Bernal $78.750 solo hicieron una compra, pero con varias boletas. Con esta lista la administración podría ofrecer un programa de fidelidad a estos 6 clientes y un incentivo de recompra a los que compraron una sola vez.

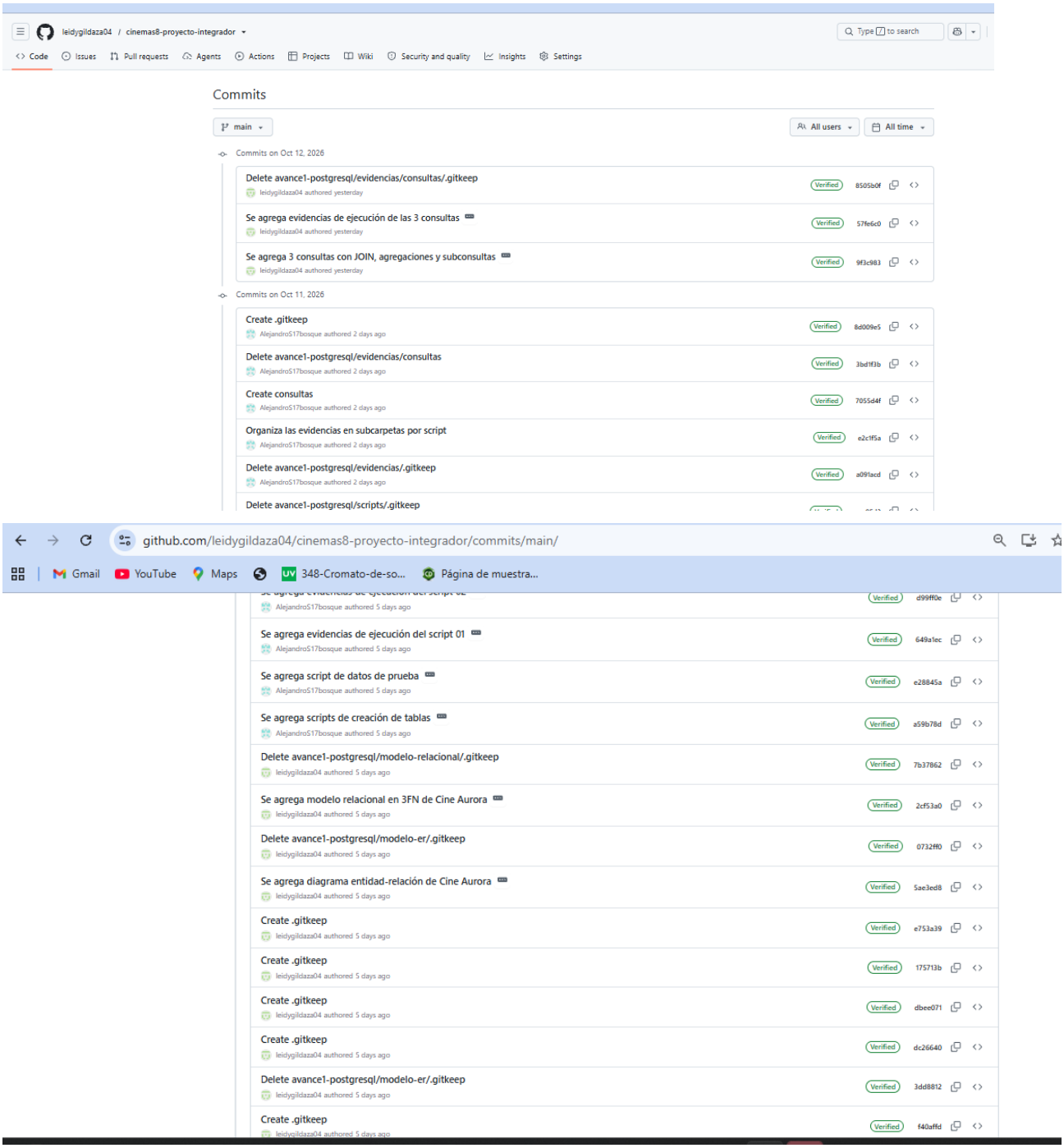
9. Repositorio y control de versiones

El trabajo está en un repositorio de GitHub, en la carpeta avance1-postgresql/, con estos archivos:

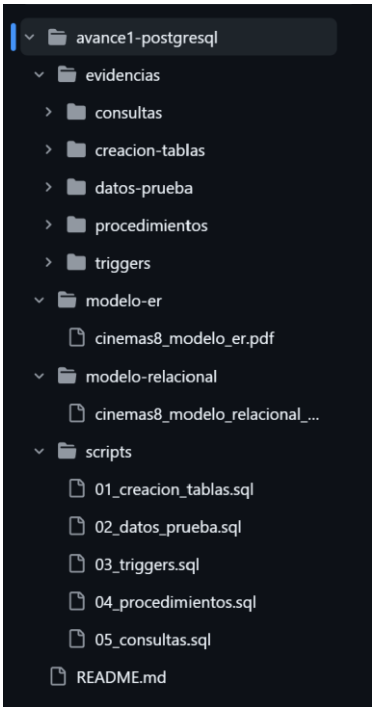
Archivo	Contenido
evidencias/	Capturas de ejecución organizadas por subcarpetas (consultas, creación de tablas, datos de prueba, procedimientos y triggers).
modelo-er/	Carpeta que contiene el diagrama y archivo del Modelo Entidad-Relación (cinemas8_modelo_er.pdf).
modelo-relacional/	Carpeta que contiene el diagrama y archivo del Modelo Relacional Normalizado en 3FN (cinemas8_modelo_relacional_3fn.pdf).
scripts/	Carpeta que contiene los scripts SQL del proyecto:
└─ 01_creacion_tablas.sql	Creación de las 11 tablas, llaves, restricciones e índices.
└─ 02_datos_prueba.sql	Datos de prueba coherentes.
└─ 03_triggers.sql	8 triggers con sus respectivas pruebas.
└─ 04_procedimientos.sql	3 procedimientos almacenados con sus pruebas.
└─ 05_consultas.sql	3 consultas SQL.
README.md	Archivo con la descripción general y documentación del repositorio.
avance1-informe.pdf	Este informe final en formato PDF.

Enlace al repositorio	https://github.com/leidygildaza04/cinemas8-proyecto-integrador/tree/main/avance1-postgresql

9.1 Commits por integrante



Captura 30. Estructura de la carpeta avance1-postgresql/ en GitHub



AVANCE 2 — Capa Documental (MongoDB) y Fundamentos Teóricos

1. Introducción

En este segundo avance construimos la capa documental de Cine Aurora con MongoDB y la dejamos preparada para integrarse con el núcleo transaccional que ya hicimos en PostgreSQL (Avance 1). La idea central es que cada motor guarde lo que mejor sabe manejar: PostgreSQL se encarga de las ventas, funciones, salas y demás datos que no pueden fallar ni quedar a medias, y MongoDB guarda la información descriptiva y cambiante del cine: fichas técnicas, requisitos de proyección, reseñas e incidentes.

Cine Aurora es un cine de 8 salas en Bogotá D.C. (2 VIP, 2 en formato 3D y 4 en 2D) que combina cine de autor con estrenos comerciales. Esa mezcla genera justo el tipo de datos que se lleva mal con tablas rígidas: una película puede tener varias versiones, idiomas y clasificaciones; una función IMAX o Dolby Atmos exige requisitos distintos a una 2D; las reseñas llegan en gran volumen y cada tipo de incidente se describe con campos diferentes.

Este documento reúne el diseño de las cuatro colecciones con la justificación de cada decisión de embebido o referenciado, el diagrama de arquitectura políglota, las validaciones con $jsonSchema, los datos de prueba, las consultas, las capturas de ejecución y la justificación teórica con ACID, BASE y CAP aplicada al dominio del cine. Todo el código está en la carpeta avance2-mongodb/ del repositorio.

2. Arquitectura de persistencia poliglota

El sistema usa dos motores a la vez. PostgreSQL guarda las 11 tablas del núcleo transaccional (películas, salas, funciones, ventas, espectadores, staff, etc.) y MongoDB guarda las 4 colecciones documentales en la base de datos cine_aurora. Los dos se conectan por identificadores compartidos: cada documento de MongoDB lleva el id de la fila de PostgreSQL a la que se refiere (id_pelicula, id_funcion, id_sala, id_espectador, id_staff, id_actor_director).

Es importante aclarar que estas son referencias lógicas, no llaves foráneas: MongoDB no verifica que el id exista en PostgreSQL, esa validación la hace la aplicación. Esto lo retomamos en la sección de CAP y BASE porque es una de las consecuencias de usar dos motores.

Colección (MongoDB)	Identificador compartido	Tabla de PostgreSQL
fichas_tecnicas_peliculas	id_pelicula, id_actor_director (dentro de elenco)	peliculas, actores_directores
requisitos_proyeccion	id_funcion, id_sala	funciones, salas
resenas_peliculas	id_pelicula, id_espectador	peliculas, espectadores
reportes_incidentes	id_sala, id_funcion, id_staff	salas, funciones, staff

Lo que hicimos de diseño fue que acá embebemos lo que nace, vive y se consulta junto con su documento y que no tiene sentido por sí solo. Referenciamos (con el id de PostgreSQL) todo lo que es una entidad con vida propia en el núcleo relacional, para no duplicar datos que podrían cambiar y quedar desactualizados en un lado.

2.1 Diagrama de arquitectura políglota

El diagrama muestra las entidades en PostgreSQL (izquierda), las colecciones en MongoDB (derecha) y las líneas punteadas con el identificador compartido que conecta cada colección con su tabla.

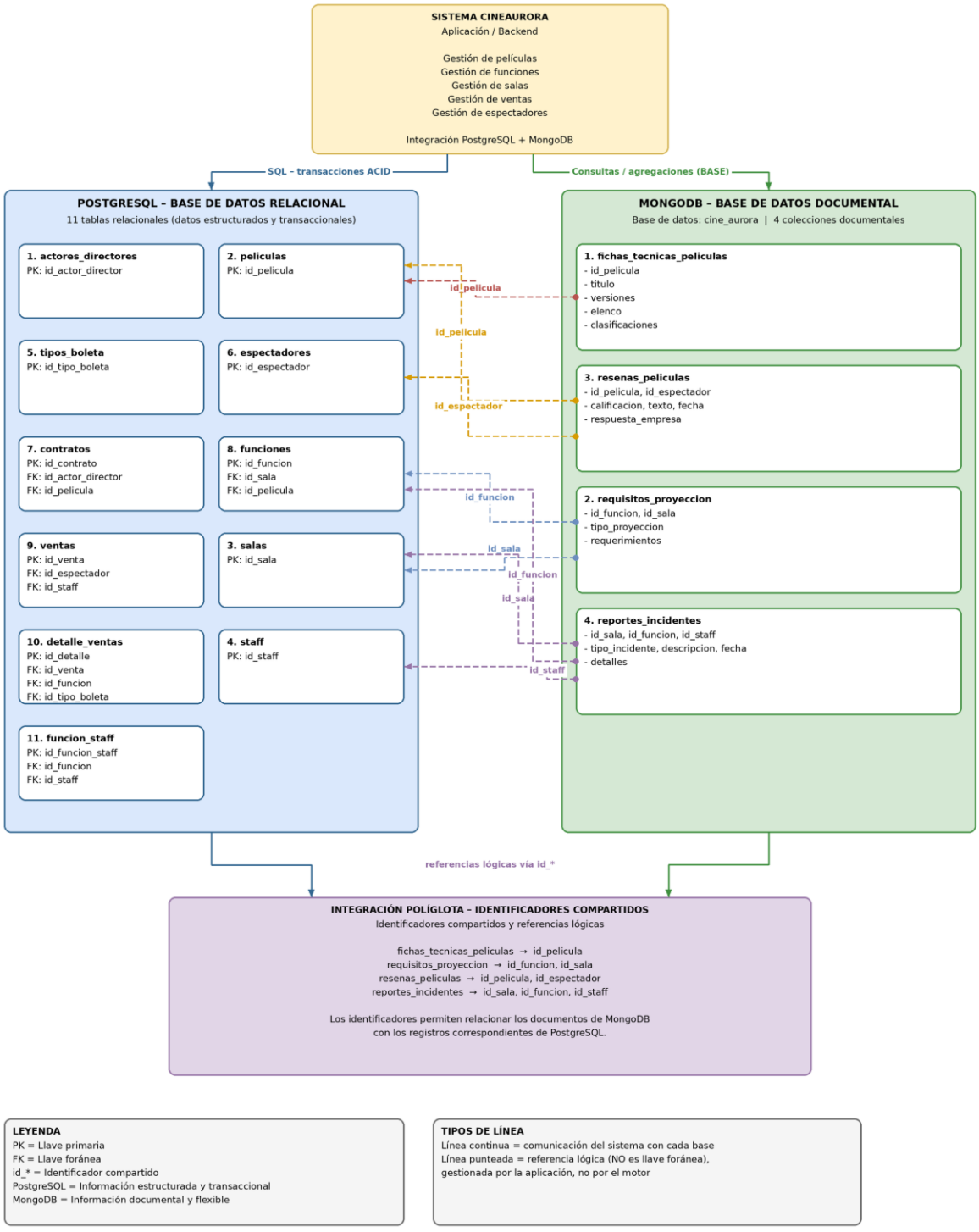


Figura 1. Arquitectura políglota de Cine Aurora (PostgreSQL + MongoDB).

3. Diseño de las colecciones

Para cada colección presentamos su estructura, la razón por la que es documental y una tabla con cada decisión de embebido o referenciado.

3.1 fichas_tecnicas_peliculas

Contenido: elenco, duración por versión, idiomas, subtítulos y clasificación por país. Por qué es documental: la estructura cambia de una película a otra: cuántas versiones tiene, en qué idiomas, con qué subtítulos y en cuántos países se clasifica. En PostgreSQL esto obligaría a crear varias tablas auxiliares (versiones, idiomas, subtítulos, clasificaciones) y a hacer joins cada vez que se quiere mostrar una ficha.

Campo	Tipo	Descripción
id_pelicula	int	Identificador compartido con peliculas (PostgreSQL)
titulo	string	Título de la película
versiones	array de objetos	formato, duracion_min, idioma, subtitulos[]
elenco	array de objetos	id_actor_director y papel
clasificaciones	array de objetos	pais y clasificacion

Elemento	Decisión	Justificación
versiones	Embebido	Se leen siempre junto con la ficha, son pocas por película y no existen fuera de ella. Cada versión tiene su propia duración, idioma y subtítulos.
clasificaciones	Embebido	Dato pequeño, sin vida propia; la cantidad de países varía por película, algo que un array resuelve sin columnas vacías.
elenco	Embebido con referencia	Se embebe el papel, pero el artista se referencia con id_actor_director. No copiamos nombre ni contrato: si algo cambia en actores_directores no queda desactualizado aquí.
id_pelicula	Referenciado	La película como entidad (título base, género, año) se administra en PostgreSQL; la ficha solo guarda el id compartido.

Ejemplo de documento:

```
{
  id_ficha: 1, id_pelicula: 1,
  titulo: "El Último Tranvía de Chapinero",
  versiones: [{ formato: "Digital 3D", duracion_min: 91,
               idioma: "Inglés", subtitulos: ["Español"] }],
  elenco: [{ id_actor_director: 1, papel: "Actor principal" }],
  clasificaciones: [{ pais: "Colombia", clasificacion: "Mayores de 12 años" }]
}
```

3.2 requisitos_proyeccion

Contenido: requerimientos técnicos por función especial (IMAX, 3D, Dolby Atmos, 2D). Por qué es documental: los requisitos varían por función y formato. Una función 3D necesita gafas y proyector 3D; una Dolby Atmos necesita un sistema de sonido específico; una IMAX, resolución 4K. Si se modelara con columnas, la mayoría quedarían en NULL para cualquier función dada.

Campo	Tipo	Descripción
id_funcion	int	Identificador compartido con funciones (PostgreSQL)
id_sala	int	Identificador compartido con salas (PostgreSQL)
tipo_proyeccion	string	2D, 3D, IMAX o Dolby Atmos
requerimientos	objeto	proyector, sonido, gafas_3d, resolucion (varían según el tipo)

Elemento	Decisión	Justificación
requerimientos	Embebido	Es un objeto que solo tiene sentido dentro de esa función y cuya forma depende del tipo de proyección. Se consulta siempre junto con el tipo, nunca por separado.
id_funcion	Referenciado	Los horarios, la película y el estado de la función viven en PostgreSQL, donde se validan solapamientos. No los copiamos para no tener dos fuentes de verdad.
id_sala	Referenciado	La capacidad y el tipo de sala los administra PostgreSQL (tabla salas); aquí solo se guarda el id.

Ejemplo de documento:

```
{
  id_funcion: 2, id_sala: 2, tipo_proyeccion: "3D",
  requerimientos: { proyector: "Proyector 3D", sonido: "Estéreo",
                    gafas_3d: true, resolucion: "2K"}
}
```

3.3 resenas_peliculas

Contenido: texto libre, calificación, fecha y respuesta de la empresa. Por qué es documental: el volumen es alto y el esquema poco uniforme (hay reseñas largas, cortas, con o sin respuesta). Es información que crece sin parar y que no participa en las transacciones de venta.

Campo	Tipo	Descripción
id_pelicula	int	Identificador compartido con peliculas
id_espectador	int	Identificador compartido con espectadores
calificacion	int / double	Valor entre 1 y 5
texto	string	Opinión libre del espectador
fecha	date	Fecha de la reseña
respuesta_empresa	objeto	respondida (booleano) y mensaje

Elemento	Decisión	Justificación
respuesta_empresa	Embebido	Hay máximo una por reseña, pertenece solo a esa reseña y nunca se consulta sola.
Reseñas dentro de la ficha de la película	No embebidas (documento propio)	El número de reseñas por película no tiene límite. Meterlas en la ficha haría crecer el documento hasta acercarse al tope de 16 MB de MongoDB y cada reseña nueva obligaría a reescribir un documento enorme. Cada reseña es un documento independiente.
id_pelicula	Referenciado	Permite agrupar reseñas por película sin duplicar datos de la película.
id_espectador	Referenciado	Los datos personales (nombre, correo) quedan solo en PostgreSQL; la reseña no los copia.

Ejemplo de documento:

```
{
  id_pelicula: 2, id_espectador: 2, calificacion: 3,
  texto: "Buena historia y actuaciones.",
  fecha: ISODate("2026-10-02"),
  respuesta_empresa: { respondida: true,
                       mensaje: "Gracias por compartir tu opinión."}
}
```

3.4 reportes_incidentes

Contenido: fallas técnicas, quejas de servicio e incidentes de seguridad. Por qué es documental: los campos son distintos según el tipo de incidente. Es la colección donde más se aprovecha la flexibilidad de MongoDB: tres tipos de incidente conviven en la misma colección, cada uno con su propio objeto de detalles.

Campo	Tipo	Descripción
id_sala	int	Identificador compartido con salas (obligatorio)
id_funcion	int o null	Identificador compartido con funciones (opcional)
id_staff	int o null	Identificador compartido con staff (opcional)
tipo_incidente	string	Falla técnica, Queja o Seguridad
descripcion	string	Descripción del incidente
fecha	date	Fecha del incidente
detalles	objeto	Campos que cambian según el tipo de incidente

Tipo de incidente	Contenido del objeto detalles
Falla técnica	equipo, gravedad
Queja	motivo, canal
Seguridad	zona, accion

Elemento	Decisión	Justificación
detalles	Embebido (forma variable)	Es la razón principal de usar MongoDB aquí. En PostgreSQL tocaría una tabla por tipo o muchas columnas casi siempre vacías. Como objeto embebido, cada incidente guarda solo los campos que le corresponden.
id_sala	Referenciado	La sala es una entidad de PostgreSQL; el incidente solo guarda el id.
id_funcion, id_staff	Referenciados y opcionales	Un incidente puede ocurrir fuera de una función o sin personal asignado, por eso aceptan null. El detalle de la función y del empleado no se copia.

Ejemplo de documento:

```
{
  id_sala: 1, id_funcion: 1, id_staff: 1,
  tipo_incidente: "Falla técnica",
  descripcion: "Problema con el sistema de proyección.",
  fecha: ISODate("2026-10-01"),
  detalles: { equipo: "Proyector", gravedad: "Media" }
}
```

4. **Validación con $jsonSchema**

Definimos validadores $jsonSchema en las 4 colecciones (el mínimo pedido eran 2), con validationLevel "strict" y validationAction "error", de modo que MongoDB rechaza cualquier documento que no cumpla. Los validadores exigen los identificadores compartidos y los tipos de dato, pero dejamos libre el contenido interno de requerimientos y detalles, porque justamente ahí es donde la estructura debe poder variar.

Colección	Campos obligatorios	Reglas adicionales
fichas_tecnicas_peliculas	id_pelicula, titulo, versiones	Cada versión exige formato y duracion_min (enteros y strings según corresponde)
requisitos_proyeccion	id_funcion, id_sala, tipo_proyeccion	requerimientos debe ser un objeto
resenas_peliculas	id_pelicula, id_espectador, calificacion, texto	calificacion entre 1 y 5; fecha de tipo date
reportes_incidentes	id_sala, tipo_incidente, descripcion, fecha	id_funcion e id_staff aceptan int o null; detalles debe ser objeto

Fragmento del script 01_creacion_colecciones.js (colección resenas_peliculas):

```
> db.createCollection("resenas_peliculas", {
    validator: { $jsonSchema: {
      bsonType: "object",
      required: ["id_pelicula", "id_espectador", "calificacion", "texto"],
      properties: {
        id_pelicula:   { bsonType: "int" },
        id_espectador: { bsonType: "int" },
        calificacion:  { bsonType: ["int", "double"], minimum: 1, maximum: 5 },
        texto:         { bsonType: "string" },
        fecha:         { bsonType: "date" },
        respuesta_empresa: { bsonType: "object" }
      }
    } },
    validationLevel: "strict", validationAction: "error"
  });
< { ok: 1 }
```

5. **Scripts y datos de prueba**

Los scripts están en la carpeta scripts/ y se ejecutan en este orden desde mongosh sobre la base cine_aurora:

Script	Qué hace
01_creacion_colecciones.js	Crea las 4 colecciones con su validador $jsonSchema.
02_datos_prueba.js	Inserta 30 documentos en cada colección con insertMany (120 en total).
03_consultas.js	Ejecuta 6 consultas con find y 3 agregaciones con aggregation pipeline.

Cada colección tiene 30 documentos, que cumple el mínimo pedido. Los identificadores compartidos que usamos en los datos corresponden a registros que existen en los datos de prueba de PostgreSQL del Avance 1 (por ejemplo, 15 películas, 8 salas, 30 funciones, 30 espectadores, 15 empleados y 40 artistas), de modo que las referencias lógicas apuntan a filas reales.

Colección	Documentos
fichas_tecnicas_peliculas	30
requisitos_proyeccion	30
resenas_peliculas	30
reportes_incidentes	30
Total	**120**

6. Consultas en MongoDB

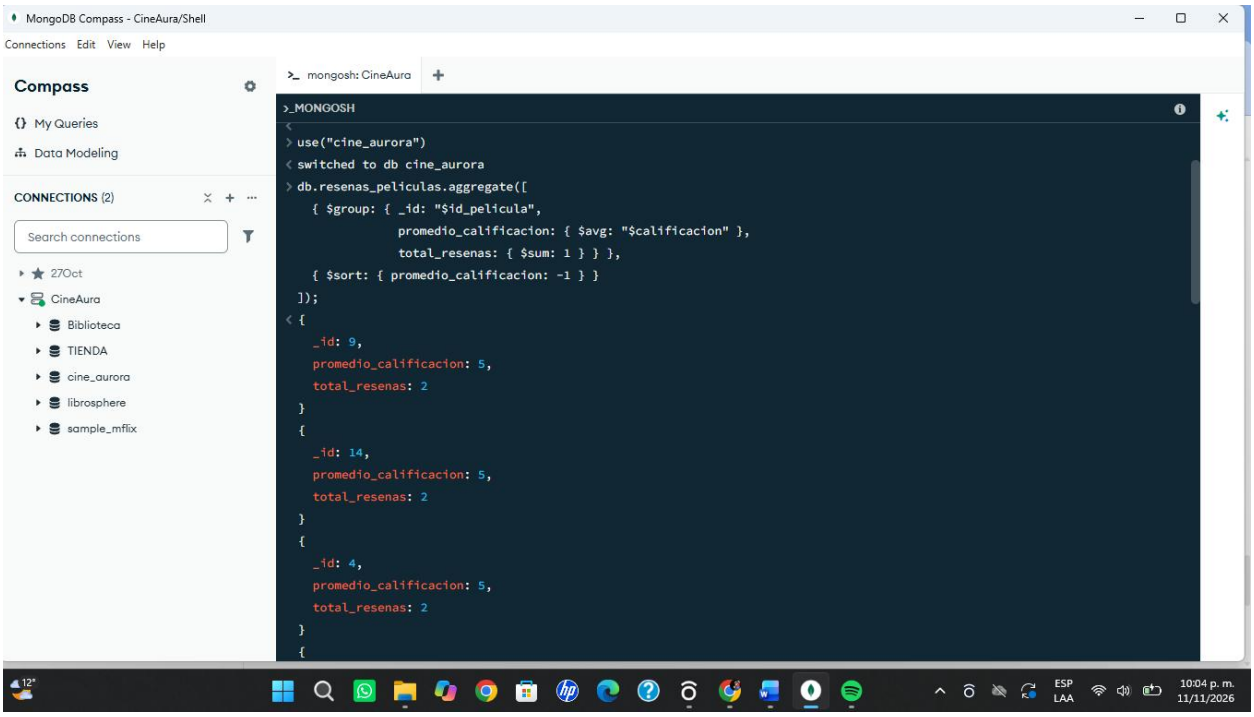
Implementamos 9 consultas: hicimos 6 con find y 3 con aggregation pipeline (el mínimo era 4, con al menos 2 agregaciones).

6.1 Consultas con find

N. º	Qué responde	Consulta
1	Fichas técnicas de una película	db.fichas_tecnicas_peliculas.find({ id_pelicula: 1 })
2	Funciones con proyección 3D	db.requisitos_proyeccion.find({ tipo_proyeccion: "3D"})
3	Reseñas con calificación alta (4 o más)	db.resenas_peliculas.find({ calificacion: {$gte: 4}})
4	Incidentes de seguridad	db.reportes_incidentes.find({ tipo_incidente: "Seguridad"})
5	Reseñas escritas por un espectador	db.resenas_peliculas.find({ id_espectador: 10})
6	Incidentes ocurridos en una sala	db.reportes_incidentes.find({ id_sala: 1})

6.2 Consultas con aggregation pipeline

Agregación 1: Promedio de calificaciones y total de reseñas por película. Sirve para ver qué películas están mejor valoradas; el resultado se puede cruzar en la aplicación con el título que está en PostgreSQL usando id_pelicula.



Agregación 2: Cantidad de incidentes por tipo. Permite saber si el problema principal del cine son las fallas técnicas, las quejas o la seguridad.

```
> db.reportes_incidentes.aggregate([
    { $group: { _id: "$tipo_incidente", total_incidentes: { $sum: 1 } } },
    { $sort: { total_incidentes: -1 } }
  ]);
< {
    _id: 'Falla técnica',
    total_incidentes: 10
  }
  {
    _id: 'Queja',
    total_incidentes: 10
  }
  {
    _id: 'Seguridad',
    total_incidentes: 10
  }
```

Agregación 3: Cantidad de funciones por tipo de proyección. Muestra cuántas funciones usan cada formato (2D, 3D, IMAX, Dolby Atmos), útil para planear equipos y mantenimiento.

```
> db.requisitos_proyeccion.aggregate([
    { $group: { _id: "$tipo_proyeccion", total_funciones: { $sum: 1 } } },
    { $sort: { total_funciones: -1 } }
  ]);
< {
    _id: '3D',
    total_funciones: 8
  }
  {
    _id: '2D',
    total_funciones: 8
  }
  {
    _id: 'Dolby Atmos',
    total_funciones: 7
  }
  {
    _id: 'IMAX',
    total_funciones: 7
  }
```

7. Capturas de ejecución

Las siguientes capturas corresponden a la ejecución real de los scripts sobre MongoDB (Compass, mongosh y Visual Studio Code) conectados a un clúster de MongoDB Atlas. También están en la carpeta evidencias/ del repositorio.

7.1 Creación de las colecciones (script 01)

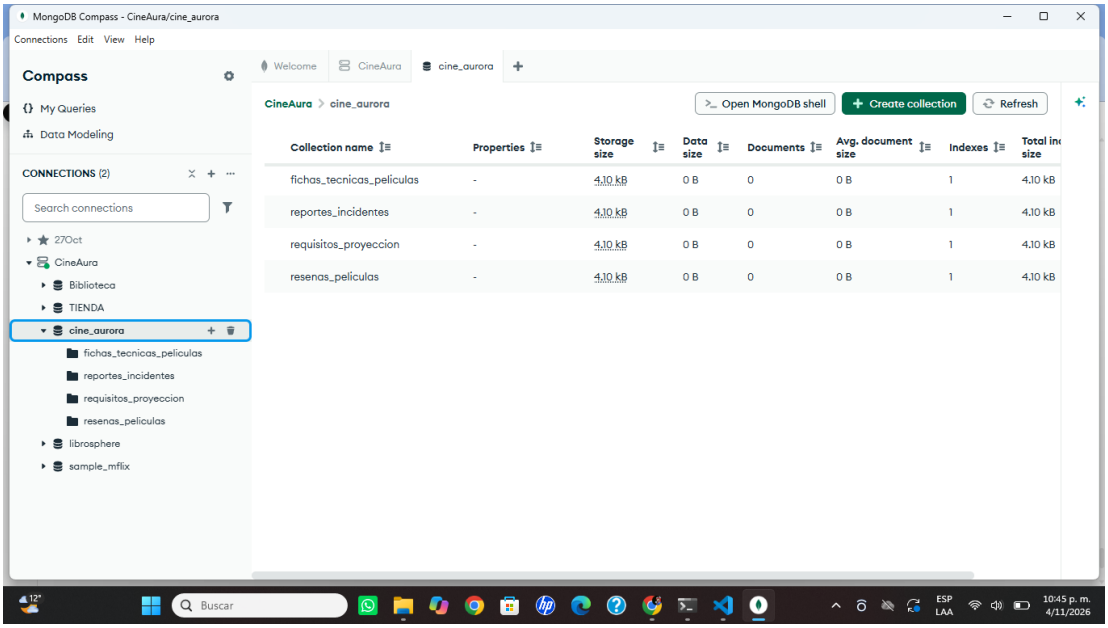


Figura 2. Las cuatro colecciones creadas en la base de datos cine_aurora (aún sin documentos).

7.2 Carga de datos de prueba (script 02)

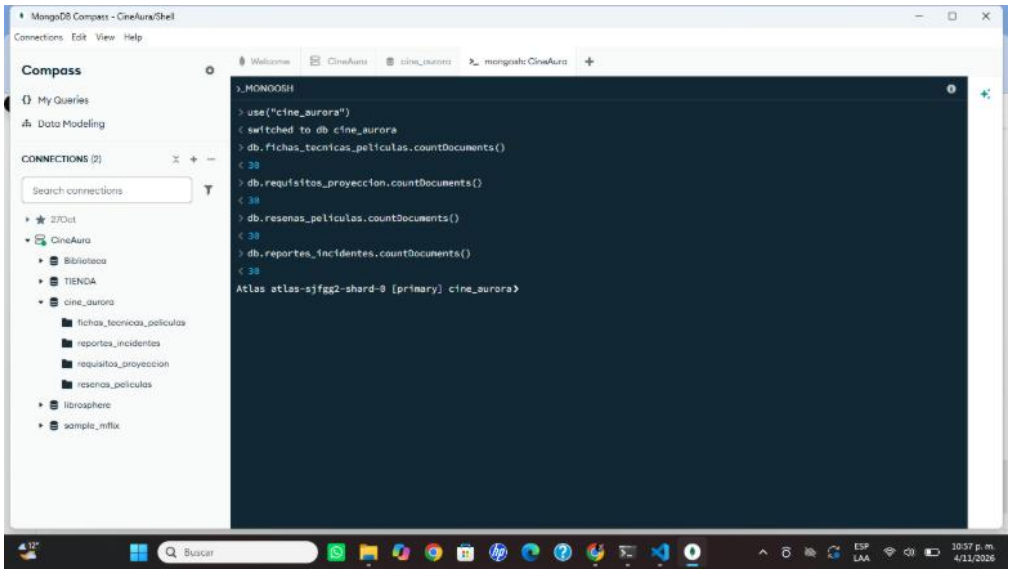


Figura 3. Conteo con countDocuments(): 30 documentos en cada una de las cuatro colecciones.

7.3 Consultas y agregaciones (script 03)

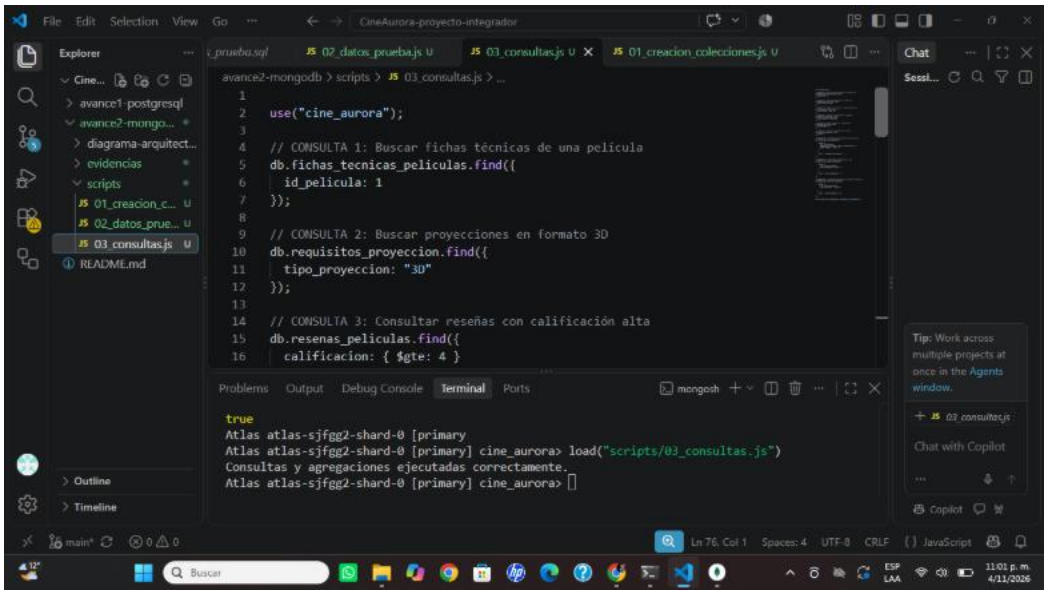


Figura 4. Ejecución de 03_consultas.js desde mongosh con load(), con el mensaje de confirmación final.

7.4 Agregación 2: incidentes por tipo (script 03)

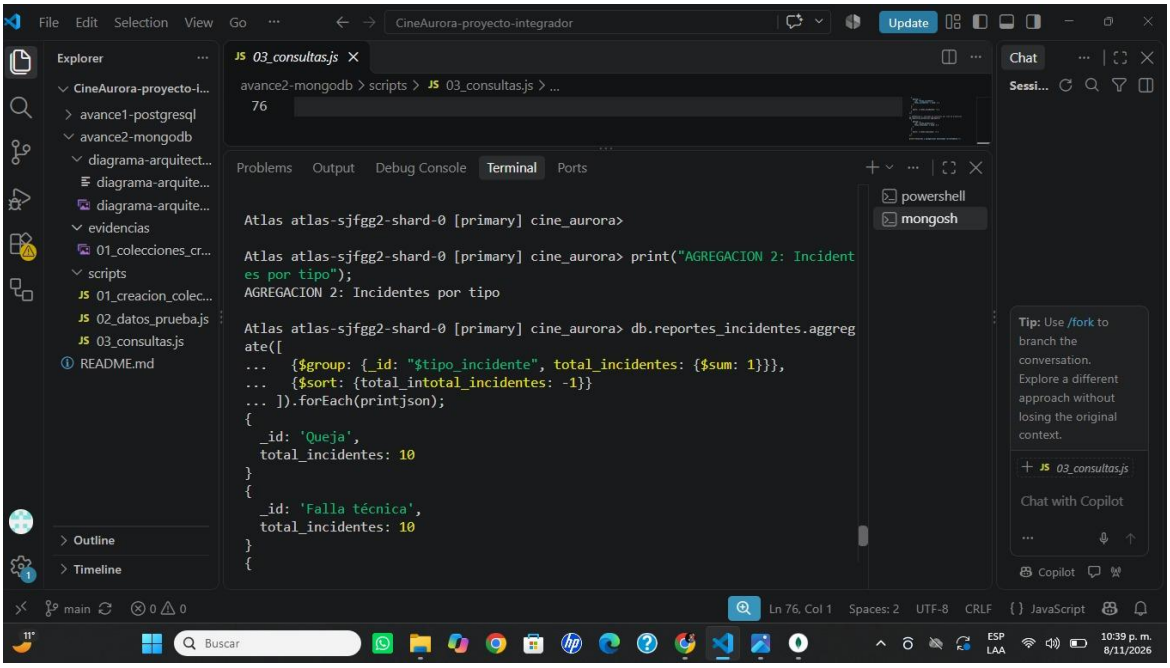


Figura 5. Resultado de la agregación 2 en mongosh: total de incidentes por tipo (reportes_incidentes).

7.5 Agregación 3: funciones por tipo de proyección (script 03)

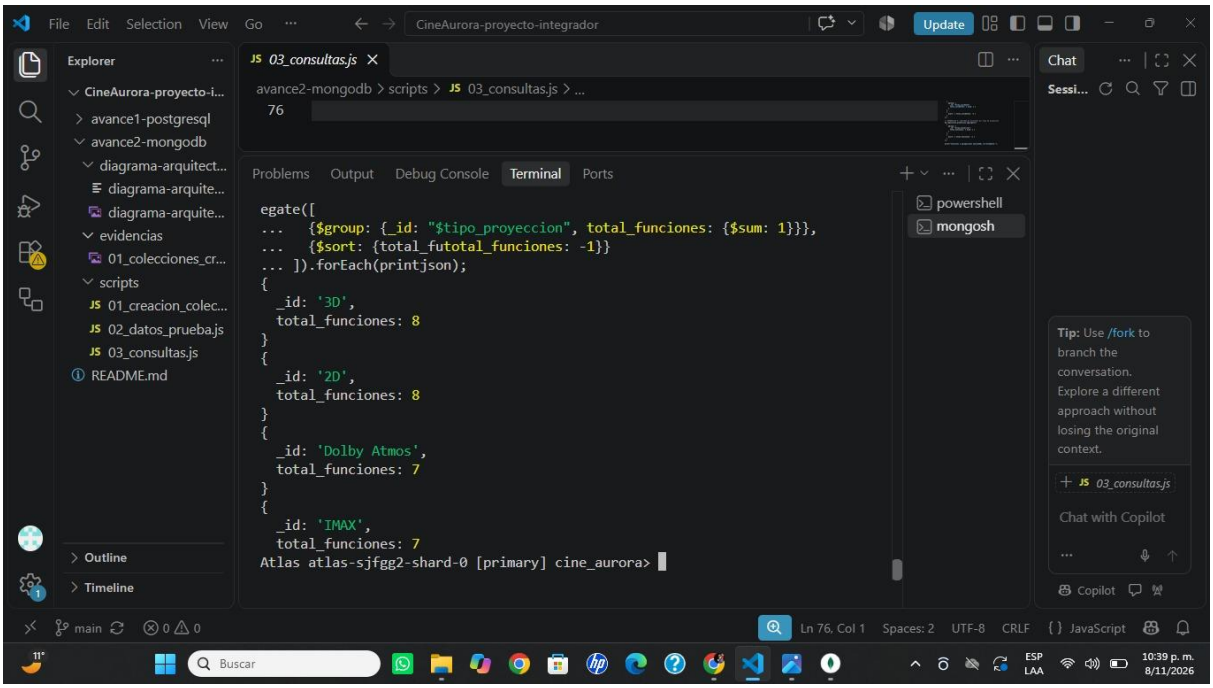


Figura 6. Resultado de la agregación 3 en mongosh: total de funciones por tipo de proyección (requisitos_proyeccion).

8. Justificación teórica ACID, BASE Y CAP

Separar los datos en dos motores no fue una decisión de moda sino de qué tan grave es que un dato esté mal o desactualizado. Una venta de boletas no puede quedar a medias ni sobrepasar el aforo; una reseña que se muestra unos segundos tarde no le hace daño a nadie. Esa diferencia es la que explican ACID y BASE.

8.1 ACID en PostgreSQL

ACID agrupa las garantías de una transacción: atomicidad, consistencia, aislamiento y durabilidad. En Cine Aurora las usa todo lo que mueve dinero o cupo, y por eso vive en PostgreSQL:

Propiedad	Cómo se aplica en Cine Aurora	Por qué es necesaria
Atomicidad	El procedimiento registrar_venta inserta la cabecera en ventas y su detalle en detalle_ventas. Si cualquier validación falla (función ya pasada, sin cupo, espectador inexistente), el RAISE EXCEPTION cancela todo.	No puede existir una venta sin boletas ni boletas sin venta; el cliente quedaría cobrado sin entrada, o con entrada sin pago.
Consistencia	Llaves primarias y foráneas, restricciones CHECK y triggers como trg_validar_capacidad (no vender más boletas que la capacidad de la sala, por ejemplo 80 en la sala Aurora), trg_calcular_precio (aplica el descuento de la categoría) y trg_validar_taquillero.	La base solo puede pasar de un estado válido a otro válido: nunca se sobrevende una función ni se registra una venta física sin un taquillero.
Aislamiento	Taquilla física y venta virtual operan al mismo tiempo sobre las mismas funciones. PostgreSQL aísla las transacciones concurrentes para que ninguna lea cambios a medias de otra.	Evita que dos compradores se lleven la última boleta a la vez. Como el cupo se calcula contando las boletas vendidas, en un caso extremo de concurrencia convendría reforzarlo bloqueando la fila de la función (SELECT ... FOR UPDATE) o con SERIALIZABLE.
Durabilidad	Una vez confirmado el COMMIT, la venta queda registrada aunque el servidor se reinicie (registro de transacciones WAL).	Si el cliente ya pagó y tiene su boleta, esa venta no se puede perder.

Esta es la razón de que ventas, boletas, funciones, salas, contratos y staff estén en PostgreSQL: son datos donde la **consistencia fuerte** vale más que la flexibilidad.

8.2 BASE en MongoDB

BASE significa Basically Available, Soft state y Eventual consistency (disponibilidad básica, estado flexible y consistencia eventual). Es el enfoque típico de las bases documentales: se prioriza que el sistema siempre responda y se acepta que, durante un tiempo corto, distintas copias o resúmenes de un dato no coincidan. En Cine Aurora lo aceptamos en MongoDB porque ninguna de las cuatro colecciones participa en el cobro ni en el control de aforo:

Colección	Dónde se acepta BASE	Por qué es tolerable
resenas_peliculas	El promedio de calificaciones (agregación 1) puede mostrarse unos segundos desactualizado frente a una reseña recién escrita. Si se leen réplicas secundarias, una reseña nueva puede tardar en aparecer.	Nadie pierde dinero ni cupo porque una reseña tarde en verse. El promedio es informativo.
reportes_incidentes	Un incidente recién reportado puede tardar en aparecer en los conteos por tipo (agregación 2).	Los reportes se usan para análisis y seguimiento, no para decidir una venta en tiempo real.
fichas_tecnicas_peliculas	Una corrección en la ficha (por ejemplo un subtítulo) puede verse con retraso en las réplicas.	Es información descriptiva que cambia poco; una demora de segundos no afecta la operación.
requisitos_proyeccion	El requisito técnico de una función puede estar desfasado un instante respecto a un cambio hecho en PostgreSQL.	El equipo técnico revisa estos requisitos antes de la función, con horas de anticipación.

Aceptar BASE también tiene un costo que hay que reconocer: como las referencias entre MongoDB y PostgreSQL son lógicas, MongoDB no impide guardar una reseña con un id_pelicula que no exista en PostgreSQL. Esa verificación queda a cargo de la aplicación, y es el precio de usar dos motores.

8.3 Teorema CAP

El teorema CAP dice que un sistema distribuido solo puede garantizar dos de tres propiedades: Consistencia (todos ven el mismo dato), Disponibilidad (siempre responde) y Tolerancia a particiones (sigue funcionando si se cae la comunicación entre nodos). Como las particiones de red no son opcionales en un sistema distribuido, la decisión real es qué se sacrifica cuando ocurren.

Motor	Posición en CAP	Qué se sacrifica	Ejemplo en Cine Aurora
PostgreSQL	CP (consistencia + tolerancia a particiones)	Disponibilidad: ante una falla o partición puede rechazar operaciones en vez de aceptarlas con riesgo.	Es preferible que la taquilla muestre un error momentáneo a que dos personas compren la misma última boleta.
MongoDB	Por defecto CP; consistencia ajustable (replica set con un nodo primario)	Si se priorizan lecturas desde nodos secundarios, se sacrifica algo de consistencia para ganar disponibilidad (consistencia eventual). Si se cae el primario, las escrituras esperan hasta que se elija uno nuevo.	Una reseña leída desde una réplica puede llegar tarde, pero la consulta de reseñas sigue respondiendo.

En resumen, PostgreSQL se queda con la consistencia porque ahí están el dinero y el aforo, y a MongoDB le exigimos disponibilidad y flexibilidad porque ahí está información que tolera ligeros desfases. Cada dato vive donde su nivel de riesgo es el adecuado, que es justamente lo que justifica la arquitectura políglota.

9. Repositorio y control de versiones

El trabajo del Avance 2 está en el mismo repositorio de GitHub, en la carpeta avance2-mongodb/, con estos archivos:

Archivo	Contenido
diagrama-arquitectura-poliglota/	Carpeta con el diagrama de arquitectura políglota (imagen .png y archivo editable .drawio).
evidencias/	Capturas de ejecución real: colecciones creadas, conteo de documentos y consultas con agregaciones.
scripts/	Carpeta que contiene los scripts de MongoDB del proyecto:
└─ 01_creacion_colecciones.js	Creación de las 4 colecciones con su validador $jsonSchema.
└─ 02_datos_prueba.js	30 documentos de prueba por colección (120 en total).
└─ 03_consultas.js	6 consultas con find y 3 con aggregation pipeline.

Enlace al repositorio	https://github.com/leidygildaza04/CineAurora-proyecto-integrador/tree/main/avance2-mongodb

9.1 Commits por integrante

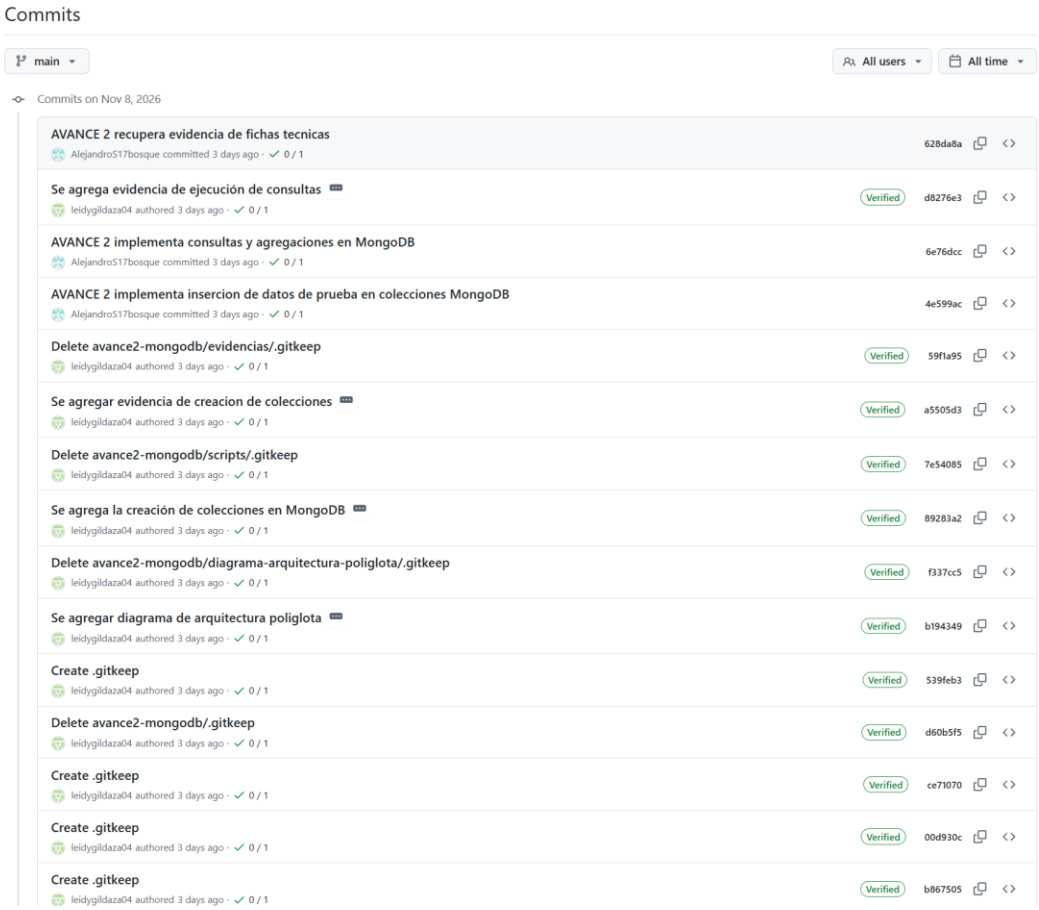
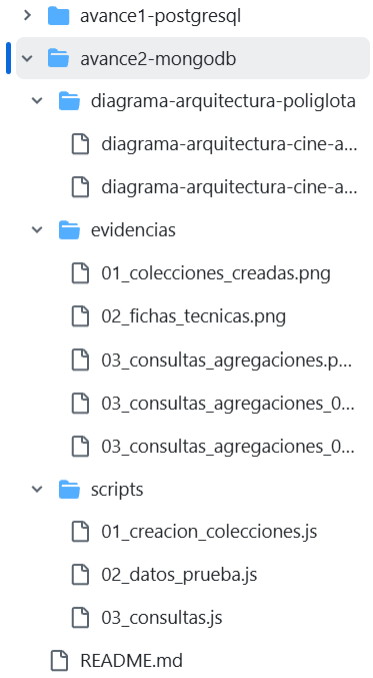


Figura 7. Estructura de la carpeta avance2-mongodb/ en GitHub

Estructura de la carpeta avance12-mongodb/ en GitHub



10. Conclusiones

- Las cuatro colecciones fichas técnicas, requisitos de proyección, reseñas e incidentes modelan información de estructura variable que en PostgreSQL requeriría muchas tablas auxiliares o columnas casi siempre vacías.
- Embebimos los datos que nacen y se consultan junto con su documento, y referenciamos con ids compartidos todo lo que es una entidad propia de PostgreSQL, evitando duplicar información.
- Aplicamos $jsonSchema en las 4 colecciones, cargamos 30 documentos en cada una y resolvimos 9 consultas (6 con find y 3 con aggregation pipeline).
- Con ACID, BASE y CAP justificamos que las ventas y el aforo permanezcan en PostgreSQL y que la información descriptiva viva en MongoDB; las colecciones quedan listas para integrarse con el núcleo relacional en la Entrega Final.